

ING SW RIASSUNTO

SW PROCESS

L'obiettivo principale del **processo software** è **produrre software** (documenti, dati e codice) con **proprietà di processo ben definite e prevedibili** e con **proprietà di prodotto adeguate**. In altre parole, non basta ottenere un software funzionante, ma è importante svilupparlo in modo controllato e prevedibile. Costituisce l'insieme delle attività con cui il sw viene sviluppato, messo in esercizio (utilizzato dagli utenti) e mantenuto ed evoluto nel tempo.

Si distinguono due tipi di proprietà:

- **Proprietà del processo (Process Properties):** descrivono **come** il software viene sviluppato. In particolare: **COSTO** (quanto costa svilupparlo), **EFFORT** (person-hours necessarie), **DURATA / PUNTUALITA'** (tempo impiegato e rispetto delle scadenze), **CONFORMITA'** agli standard e alle normative.
- **Proprietà del prodotto (Product Properties):** descrivono le **caratteristiche del software realizzato**.

Si dividono in:

- **PROPRIETA' FUNZIONALI:** descrivono cosa deve fare il sw
- **PROPRIETA' NON FUNZIONALI:** descrivono come il sw deve funzionare e i vincoli che deve rispettare: usabilità, efficienza, affidabilità, manutenibilità, sicurezza, safety. Se non rispettate il progetto fallisce.

SW PROCESS = DEVELOPMENT → OPERATION → MAINTENANCE

(DEVELOPMENT = ANALISI DEI REQUISITI + DESIGN + CODING + TESTING)

Le principali ATTIVITA' DI PRODUZIONE DEL SW sono:

- **ANALISI DEI REQUISITI (requirements engineering)** : si definiscono le esigenze del cliente e le funzionalità che il sistema dovrà offrire. "What should the sw do?"
- **Architecture and DESIGN (Progettazione):** quali componenti e come sono legati tra loro
- **IMPLEMENTATION (CODING)** : scrivere codice sorgente, eseguibile
- **INTEGRATION UNITS (verifica e validazione)**

Sono collegate logicamente perché ogni attività usa i risultati della precedente: la progettazione dipende dai requisiti, il coding dipende dal progetto, il testing dipende dal codice sviluppato e va a verificare se rispetta i requisiti. Alla fine di ogni attività si producono dei documenti. Per questo motivo, il un processo tradizionale come **WATERFALL**, le attività vengono svolte in sequenza, anche se nei processi iterativi è possibile tornare alle fasi precedenti per apportare modifiche.

(**CONWAY'S LAW:** Il sistema rispecchia la struttura dell'organizzazione. La struttura deve rispecchiare come il team lavora insieme.)

(Dobbiamo dividere un sistema complesso in dei sottomoduli con un'alta coesione e low coupling.)

(**ASTRAZIONE:** dato un problema complesso, estrae una vista semplice di esso evitando dettagli inutili e poi ragionare sull'abstract view (model).)

ATTIVITA' DI VERIFICA E VALIDAZIONE (Verification & Validation (V&V)):

Servono a verificare la qualità degli artefatti prodotti durante lo sviluppo.

VERIFICATION: controlla che ogni artefatto sia stato sviluppato correttamente rispetto alle specifiche della fase precedente ("Stiamo costruendo il prodotto nel modo giusto?").

VALIDATION: controlla che il prodotto soddisfi realmente i bisogni e le esigenze del cliente ("Stiamo costruendo il prodotto giusto?").

Le V&V si applicano a:

- **Requirements:** si verifica che i requisiti siano completi, coerenti e rappresentino le esigenze del cliente.
- **Design:** si verifica che il progetto sia consistente e soddisfi i requisiti.
- **Code:** si verifica che il codice implementi correttamente il progetto e i requisiti e sia privo di errori. Il codice compila?

Le **Ispezioni** (Inspections): Dopo ogni deliverable (documento o codice) deve esserci un'ispezione (es. una code review o una revisione dei requisiti con gli stakeholder) per bloccare i difetti prima che passino alla fase successiva

MODELLO WATERFALL: Si tratta di un modello sequenziale in cui ogni fase viene completata prima di iniziare la successiva.

- **VANTAGGI:** è semplice da pianificare e gestire, con fasi e documentazione ben definite, una struttura chiara e i goals sono noti sin da subito.
- **SVANTAGGI:** è poco flessibile, perché modificare i requisiti durante lo sviluppo è costoso. I test si fanno solo alla fine e apportare dei cambiamenti è un disastro.

MODELLO ITERATIVO: il software non viene sviluppato tutto in una volta, ma attraverso una serie di iterazioni. In ogni iterazione si sviluppa una versione funzionante del sistema, implementando solo una parte dei requisiti. Ogni iterazione comprende tutte le principali attività del processo software: analisi dei requisiti; progettazione; implementazione; testing. Al termine di ogni iterazione il cliente prova il software e fornisce un feedback, che permette di modificare i requisiti o aggiungerne di nuovi nelle iterazioni successive. In questo modo il prodotto viene migliorato progressivamente fino alla versione finale. A differenza del modello Waterfall, nel modello iterativo non è necessario conoscere fin dall'inizio tutti i requisiti: questi possono evolvere durante lo sviluppo grazie ai feedback ricevuti.

- **VANTAGGIO:** è molto flessibile e permette di gestire facilmente cambiamenti nei requisiti, riducendo il rischio di sviluppare un prodotto non adatto alle esigenze del cliente.
- **SVANTAGGIO:** richiede una buona architettura iniziale e una gestione del progetto più complessa, poiché le attività vengono ripetute ad ogni iterazione. Non è adatto a piccoli progetti.

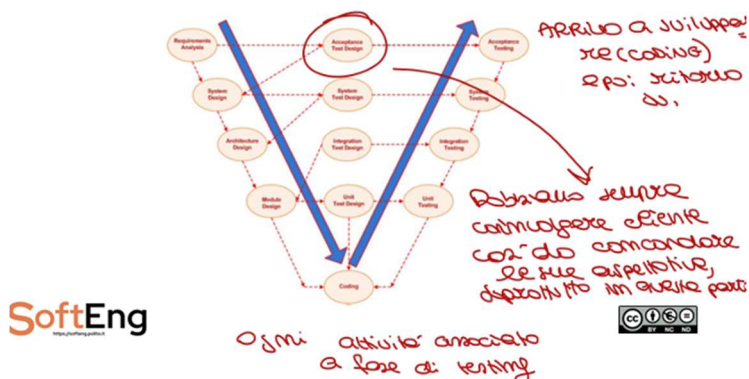
V-MODEL: Il V-Model è un'evoluzione del modello Waterfall che introduce le attività di Verification & Validation (V&V) durante tutto il processo di sviluppo. L'idea fondamentale è la progettazione anticipata dei test: per ogni fase di sviluppo viene definito il test che la verificherà. In particolare:

- durante l'Analisi dei Requisiti si progetta l'Acceptance Test, che servirà a verificare che il prodotto soddisfi le esigenze del cliente;
- durante il System Design si progetta il System Test, che verifica il funzionamento dell'intero sistema;
- durante l'Architectural Design si progetta l'Integration Test, che verifica il corretto funzionamento delle interazioni tra i moduli;
- durante il Module Design si progetta l'Unit Test, che verifica il corretto funzionamento delle singole unità software.

Successivamente si passa alla fase di Coding, che rappresenta il punto più basso della V. Terminata l'implementazione, si risale il lato destro della V eseguendo i test progettati in precedenza, partendo dagli Unit Test fino ad arrivare all'Acceptance Test.

- **VANTAGGIO:** I test vengono pianificati fin dall'inizio, rendendo più semplice individuare errori e verificare che ogni fase soddisfi quella precedente
- **SVANTAGGIO:** Come il Waterfall, è poco flessibile ai cambiamenti dei requisiti, perché richiede una pianificazione dettagliata fin dall'inizio.

The V-Model



La **manutenzione** può essere vista come una **sequenza di sviluppi** (*sequence of developments*). In ogni nuova release si svolgono le stesse attività dello sviluppo iniziale (**requirements, design, implementation e testing**), ma, a differenza del primo sviluppo, **ogni attività è vincolata da quanto fatto in precedenza**. I nuovi requisiti derivano da quelli esistenti e il nuovo design e la nuova implementazione devono rispettare le scelte architeturali e tecnologiche già adottate. Con il passare del tempo questi vincoli si accumulano, rendendo le modifiche sempre più difficili, costose e, nei casi estremi, impossibili, fino a richiedere la riscrittura dell'intera architettura.

MANAGEMENT ACTIVITIES:

- **PROJECT MANAGEMENT:** stimare e controllare il budget, assegnare il lavoro e monitorare i progressi (scomporre il progetto in varie attività e metterle in un diagramma di gannt).
- **CONFIGURATION MANAGEMENT:** identificare e storare documenti e sw units, mentre teniamo traccia delle loro relazioni e storia (tenere traccia delle varie versioni, tipo con GIT).
- **QUALITY ASSURANCE:** definire i goals di qualita, come verrà fatto il lavoro e controllare i risultati.

ISO / IEC 12207 □ norma di riferimento per un processo da seguire (per qualunque progetto). Ci dice quali processi devono esistere nel ciclo di vita del sw (primary processes (acquisition, supply...), supporting processes (documentation, conf management, quality management), organisational processes (management, improvement, infrastructure, training).

(System process: Embedded software. L'hw e sw vengono sviluppati contemporaneamente e il sw è un componente del sistema. Sviluppa tutto il sistema. Le fasi sono: system requirements (prima ancora dei requisiti dobbiamo capire cosa deve fare l'intero sistema), system design (dove si progetta l'intero sistema), sw development (sw process che è una sottofase del system process), system integration and test (dobbiamo mettere tutto insieme).

Software process: stand-alone software □ l'hw esiste già, sviluppiamo solo il sw.

Come possiamo sviluppare il sw? Ci sono 4 metodi:

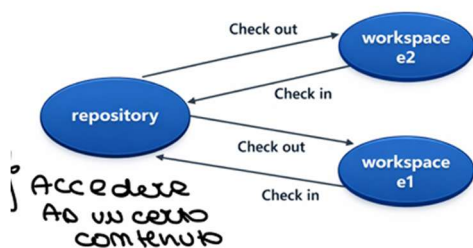
- Cowboy programming: il programmatore lavora da solo, scrive solo codice e poi vede (per progetti piccoli), non fa analisi dei requisiti, no documentazione... Molto veloce ma poco usato, per progetti grandi è inutilizzabile.
- Document based: UML, prima di programmare si producono documenti (come abbiamo fatto noi nel nostro corso), quindi doc dei requisiti, architettura, design...
- FORMALE: uso documenti, annotazioni in linguaggio formale
- AGILE: limitato uso di documenti, focus su codice e tests.

Di solito usiamo AGILE + DOCUMENT BASED.)

GIT

CONFIGURATION MANAGEMENT: gestire in modo controllato e tracciabile (per ogni fase abbiamo un documento) le MODIFICHE:

- Configuration item: insieme di file che il team decide di tracciare e versionare.
- Configuration □ insieme di Configuration item
- Repository: posto fisico in cui le varie versioni vengono salvate.
- Versioning □ capacità di ricostruire le versioni passate di un CI o configuration
- CHANGE CONTROL □ capacità di controllare gli accessi (chi può leggere/scrivere).



- checkout: accedere ad un certo contenuto
- checkin: commit (CI è stato "acceduto e già modificato")

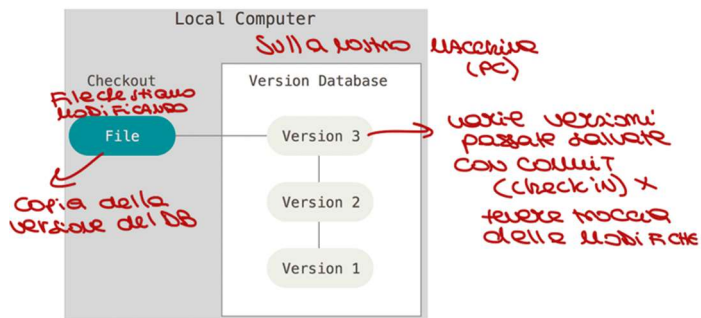
2 MODELLI:

- LOCK MODIFY UNLOCK: uno blocca i CI e gli altri non possono accederci (serializzato no conflitti).
- COPY MODIFY MERGE: + utenti lavorano in parallelo su delle copie, possono nascere dei conflitti (modifiche inconsistenti dello stesso CI da differenti users) dalle MERGE.

CMS = Configuration Management system □ sw app capace di storing e versionare i CI, storing e versionare le configurations + change control. Ripristina sia CI che configuration alle versioni

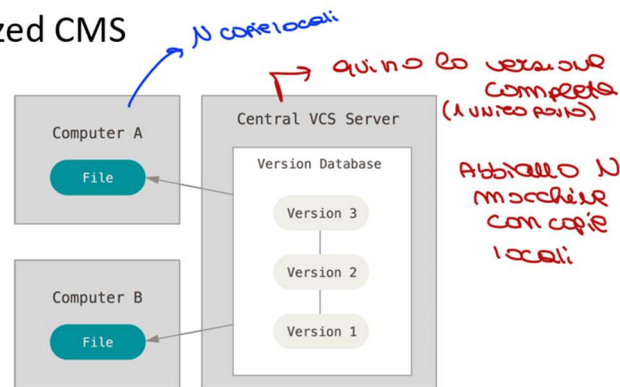
precedenti, confronta le modifiche nel tempo, controlla gli accessi e tiene traccia di chi ha modificato cosa. Può essere di 3 tipi:

- LOCALE □ tutto sulla stessa macchina , in un posto solo.



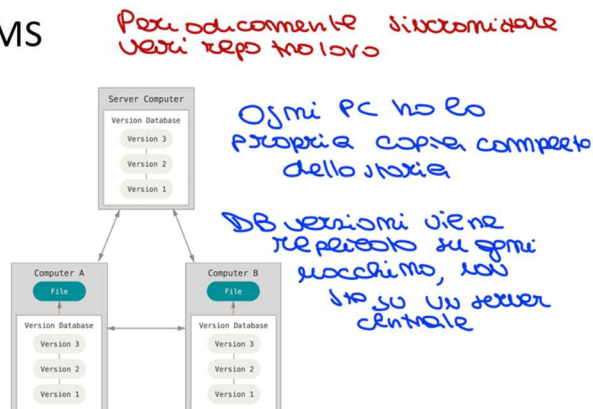
- CENTRALIZZATO □ tutti i file sul REPOSITORY remoto + copie locali

Centralized CMS



- DISTRIBUITO (GIT) □ ogni cliente ha la copia completa della repo nella sua storia.

Distributed CMS



SoftEna

STORAGE MODELS:



- DELTAS: salvo solo le modifiche, solo le differenze (occupa poco spazio).
- FULL COPIES: + spazio utilizzato, ho n copie e ogni copia è subito disponibile (ogni versione).

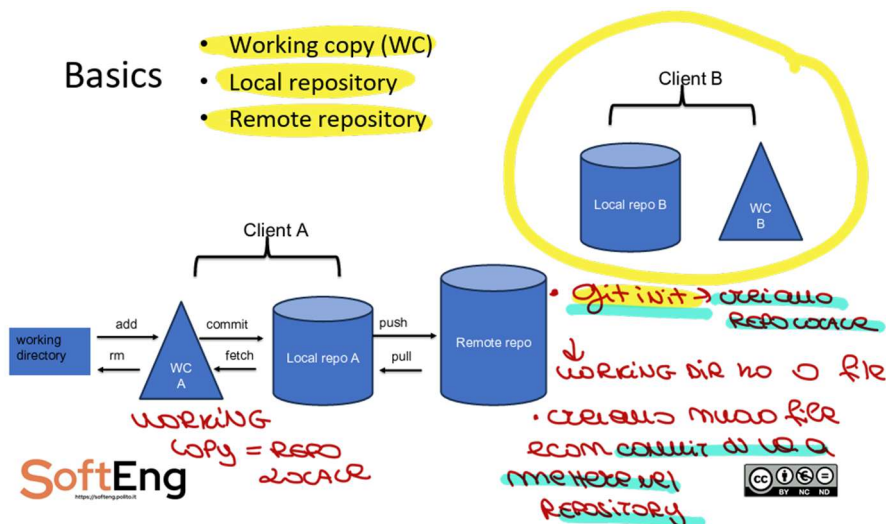
COMMIT PER GIT: SNAPSHOT (fotografia dell'intero progetto in un certo istante, sto fotografando lo stato della repo). Se un CI NON cambia versione, uso un puntatore alla versione precedente usata. (DIVERSO da DIFFERENCES in cui invece quando faccio commit viene aggiornata solo la versione dell'elemento modificata). Ho una fotografia per ogni versione.

GIT:

- Lavora per snapshot (tipo di commit)
- Distributed CMS
- Si focalizza su velocità e supporto a grandi progetti
- Integrity: fa checksum di tutti i commit → ogni cambiamento è tracciato
- Additivo: non elimina info ma le aggiunge via via.
- `git init` → creiamo la repo locale (WORKING COPY)
- `git add` → inseriamo file nel repo. Aggiungo file nella staging area, pronti per essere committati
- `git commit` → allineiamo la working copy con la repo. **Modifica repo locale**, è un'operazione atomica (se ha successo → ok, altrimenti rollback). L'integrità del repo è assicurata con checksum. Dobbiamo obbligatoriamente fornire commento con commit (con i commenti capisco la storia del progetto).
- `git push` → per mandare tutti i commit avvenuti sulla mia macchina in repo

LOCAL REPOSITORY → file e cartelle già versionate

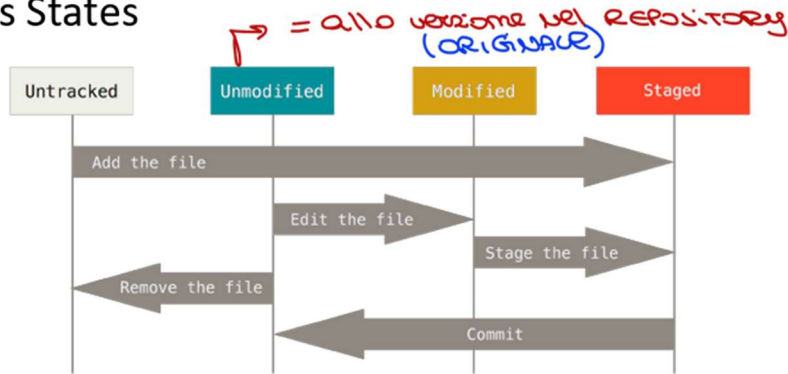
WORKING COPY (STAGING AREA): file / cartelle da committare ma non versionate.



STATO DEI CI:

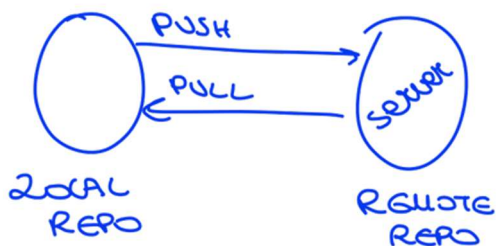
- UNTRACKED: non me ne frega delle modifiche, completamente ignorato da git come file.
- STAGED: modified / unmodified (= alla versione nel repository originale), verificabile con `git add`, sta nella staged area
- `.gitignore` → cartelle con cui diciamo a git di ignorarli perché metadati dell'ambiente di sviluppo.

CIs States



SNAPSHOT: ho tutti i file (CI) all'interno di una repo in una certa versione e ogni commit ha un hash identificativo.

LATO SERVER: Abbiamo una repo remota condivisa da tutti gli utenti. Con push (uploads snapshot da local repo a remote repo) e pull (download snapshot da remote repo a local) sincronizzo la repo locale con quella remota.

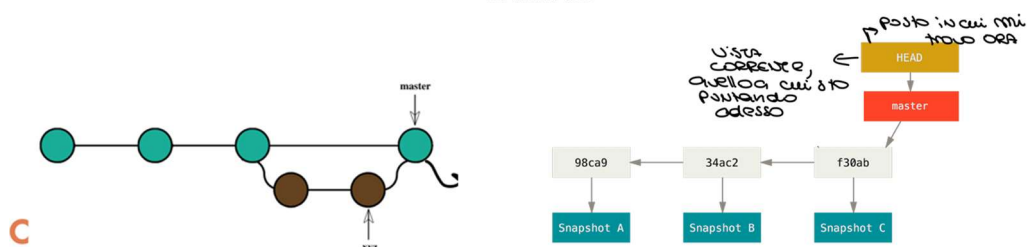


BRANCHING

Ci sono 2 modi per sviluppare un progetto:

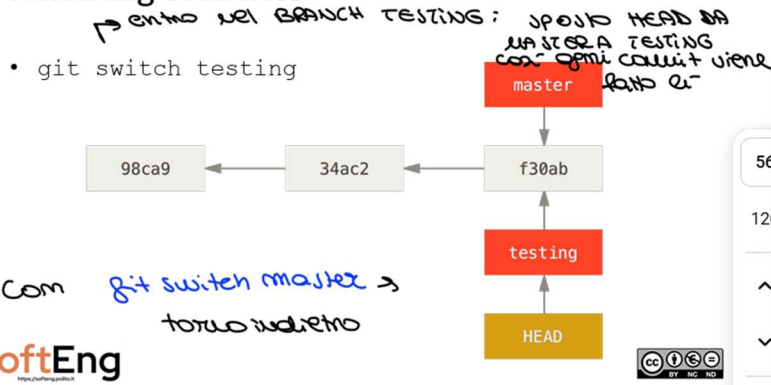
- **Lineare** □ faccio un commit dopo l'altro. Lavoriamo in sequenza, ma il problema subentra se dobbiamo cancellare delle info e tornare indietro per annullare.
- **Branching** □ Ho un main branch che rimane inalterato e per fare le modifiche non vado a modificare il flusso originale, ma un branch secondario. Se va bene poi includo la modifica con merge in flusso originale. Il commit di merge infatti unisce il parent branch con il branch figlio. Git stora un oggetto commit che contiene sia un puntatore allo snapshot dei contenuti nella staged area, sia un puntatore alla versione precedente. **HEAD** = puntatore al local branch corrente.

Branches



Con "git switch testing" ad es. spostato il puntatore al branch locale corrente (**HEAD**) sul branch figlio testing:

Switching Branches



REBASE: altro modo di fare merging, applica i cambiamenti di uno snapshot ad un altro. Mi permette di tornare ad una storia lineare.

Come si collabora su git?

- shared repository si hanno dei permessi. C'è una repo condivisa a cui possono accedere con dei permessi specifici.
- fork and pull si genero un nuovo progetto nel work space di github (mia versione di cui sono proprietario). Quando ho finito di fare le modifiche faccio una fork (una copia del progetto sotto il personal account dello sviluppatore). Poi faccio una pull request e aspetto che sia approvata dal progetto padre (divento contributors del progetto padre). Si tratta di una merge request in gitlab. Non possiamo scrivere direttamente nel main branch le modifiche ma facciamo le modifiche sul nostro branch e includiamo tutto il nostro lavoro con una merge request.

GIT

FLOW

= **modello** di gestione dei branch (branching model) nei repo per facilitare lo sviluppo sw. Definisce il work flow. E' utile per separare il codice sviluppato (stabile) da quello in produzione. Permette di sviluppare molteplici features in parallelo. Prima di fare release per garantire qualità e stabilità integriamo un branch di testing. Usiamo dei tag per tenere traccia delle varie versioni. Dobbiamo usarlo sempre!



Se ad esempio dobbiamo fare delle consegne periodiche (modello AGILE) separiamo codice di sviluppo da quality assurance ecc...

- **main branch**
 - Production-ready code
- **dev branch**
 - Base for new features
- **QA branch**
 - Testing and integration branch

BRANCH STABILI / ASSICURATI

= Continuumo a vivere
non il ciclo di vita del
prodotto

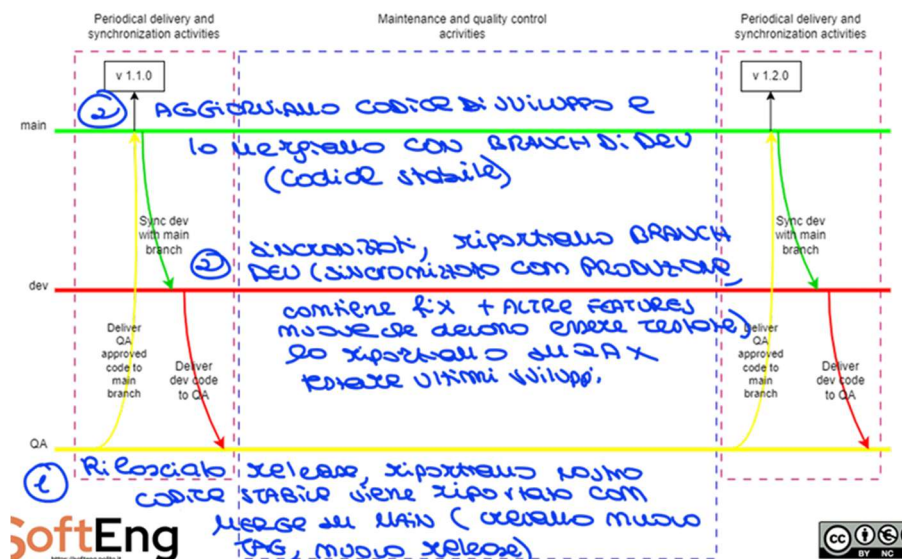
⇒ non devono mai sparire

MAIN BRANCH: contiene il codice di produzione, deve essere stabile e pronto per essere applicato, contiene codice approvato e testato. Su questo creiamo dei branch paralleli (effimeri come hotfix) per correggere i bugs di sviluppo.

DEV BRANCH: branch di sviluppo da cui si creano branch effimeri feature/branches ...

Periodicamente viene mergiato in QA per integration testing.

QA BRANCH: branch stabile usato per quality assurance: facciamo regression testing per garantire che le features esistenti funzionino e integration testing per verificare che il nuovo codice non rompa il sistema. Se un bug viene trovato per correggere: bugfix/branch...

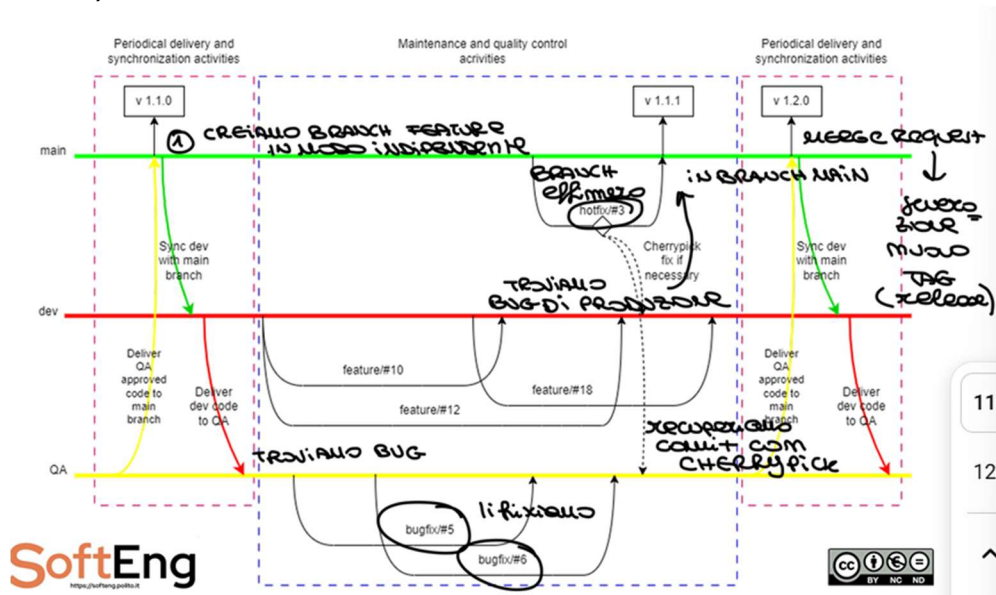


BRANCH EFFIMERI: branch temporanei creati solo per sviluppare una singola modifica e poi eliminati dopo il merge. Durano solo il tempo necessario allo sviluppo (*short lived*). Non si fanno commit direttamente sui branch principali (main, dev QA), perché potremmo rompere il codice usato da altri. Con i branch temporanei ogni sviluppo è isolato dagli altri. Quando abbiamo finito creiamo una merge request (pull request) e questo permette di fare code review, di mantenere il codice principale pulito e di fare arrivare sul main solo le modifiche validate e approvate.

TIPI DI BRANCH EFFIMERI:

- **feature/** aggiunge una nuova funzionalità richieste (branch viene creato da dev e una volta che è stato approvato e mergiato, viene cancellato).
- **bugfix/** per correggere un bug normale in fase di testing. Viene creato direttamente da QA

- hotfix/□ per correggere un bug urgente direttamente in produzione (creato direttamente da main, poi il bug viene fixato e testato. Viene creato un nuovo tag e release sul main branch).



GIT ISSUE = ticket aperto in repo per usi futuri, per bug, a cui vengono associati automaticamente branch e merge request.

Project Management

Il **Project Management** ha il compito di pianificare, organizzare e controllare il progetto software durante tutto il suo ciclo di vita. In particolare risponde a tre domande fondamentali:

- **How much?** → stima e monitoraggio di effort e costi;
 - **When is it ready?** → pianificazione e monitoraggio dei tempi;
 - **Who does what?** → organizzazione del team, allocazione delle risorse e controllo delle attività.
-
- **Resource:** una risorsa utilizzata nel progetto, ad esempio una persona o uno strumento (tools).
 - **Activity:** il tempo impiegato da una risorsa per svolgere un task definito e coerente.
 - **Phase:** un insieme di attività correlate.
 - **Milestone:** un evento o condizione chiave del progetto che influenza le attività successive. Se ho problemi qui, ho problemi dopo (no ritardi in questa fase)
 - **Deliverable:** un prodotto finale o intermedio del processo, ad esempio documento dei requisiti, prototipo, codice o test case. Può essere interno o esterno e avere valore contrattuale.

Ci sono 2 approcci:

- **WATERFALL** (tradizionale) → il progetto viene pianificato completamente all'inizio. Appena finita la raccolta dei requisiti si costruisce l'intero piano del progetto. Si tratta di una pianificazione sequenziale, ogni fase termina prima della successiva

(requisiti → analisi → progettazione → implementazione → test → consegna). Funziona se i requisiti sono stabili e se i progetti sono grandi, perché non si adatta bene ai cambiamenti continui. Definisco i requisiti, faccio una stima per ogni requisito dell'effort necessario, definisco GANTT (GRAFICO posizionamento attività nei vari mesi) e poi sviluppo.

- AGILE (iterativo) → i requisiti non sono stabili, possono cambiare continuamente (dinamici), quindi non si costruisce un piano unico. Il progetto evolve sempre. Ogni iterazione (piccolo ciclo di sviluppo) possiede il proprio piano, che viene aggiornato all'iterazione successiva. È adatto ad ambienti dinamici e progetti piccoli o in continua evoluzione.

METRICHE: process measures (time, effort, cost, faults, failures, changes, productivity), product measures (functionality, size, price...)

Aumentando il numero di risorse diminuire il tempo di calendario necessario per completare una stessa attività (quando il lavoro è parallelizzabile).

Misura	Che cosa misura	Formula / Unità	Concetti chiave da ricordare	Esempio
Calendar Time	Tempo necessario per completare il progetto o un'attività, tempo trascorso nel calendario	Giorni, settimane, mesi	• Relativo: tempo dall'inizio del progetto (es. Task A termina al giorno 20). • Absoluto: date di calendario (es. 1 settembre - 30 ottobre). • Nel tempo assoluto bisogna considerare ferie, festività, weekend, assenze.	Un task dura 10 giorni lavorativi ma occupa 14 giorni di calendario.
Effort	Quantità di lavoro necessaria per completare task	Effort = n. risorse × durata (person-hours, person-days, person-months)	Non misura il tempo trascorso ma il lavoro totale svolto dalle persone.	3 sviluppatori × 40 ore = 120 person-hours
Staff Cost	Costo del personale impiegato	Staff Cost = Effort × costo orario medio	Considera solo il costo del team di sviluppo.	120 h × 40 €/h = 4.800 €
Cost (Supplier)	Costo sostenuto da chi sviluppa il software	Personale + Overhead (costo che l'azienda spende per tenere insieme le sedi) + HW (piattaforme) + SW (licenze)	Dal punto di vista del fornitore del sw (chi costruisce il sw) .	Costo reale per produrre il software.
Price (User)	Prezzo pagato dal cliente, da chi compra il sw	TCO (Total Cost of Ownership)	Dal punto di vista dell' utente che acquista il software. Non considera solo il prezzo iniziale ma tutto il ciclo di vita. Può essere BEFORE ACQUISITION (consulta analisi di mercato, scelta fornitori...), ACQUISITION (costo del fornitore + profitto), AFTER ACQUISITION (deployment cost come training, installazione), DISMISSAL (costi di disinstallazione, backup..)	Acquisto di un ERP aziendale.

Size	Dimensione del software prodotto	LOC (lines of code)→ semplice ma approssimativo, size of documentation (numero di pag), size of tests (numero dei test cases), size of entire project (function points)	Misura quanto è grande il progetto.	—
Productivity	Efficienza dello sviluppo	Productivity = Output / Effort	Non è costante; varia con esperienza, complessità, strumenti e team.	—
Productivity (LOC)	Produttività rispetto al codice	LOC / Effort	Quante linee di codice vengono prodotte per unità di sforzo.	500 LOC / person-day
Productivity (FP)	Produttività rispetto alle funzionalità	FP / Effort	Misura più affidabile delle LOC perché indipendente dal linguaggio. Punteggio assegnato ad ogni requisito).	5 FP / person-day

STIMA DEI COSTI, 2 STRATEGIE:

Parkinson's Law: si parte dalle **risorse disponibili** (tempo e budget) e il lavoro si adatta ad esse.

Pricing to Win: si parte dal **budget del cliente** e si cerca di realizzare il progetto con quella cifra, anche se non rappresenta il costo reale di sviluppo.

VANTAGGI PARKINSON'S LAW: il progetto rimane entro il budget (svantaggio: il prodotto potrebbe risultare incompleto o con priorità gestite male..

Vantaggio PRICE TO WIN: aumenta la probabilità di ottenere il contratto. **Svantaggio:** il prezzo potrebbe non coprire i costi reali, con conseguente riduzione del profitto o della qualità del prodotto.

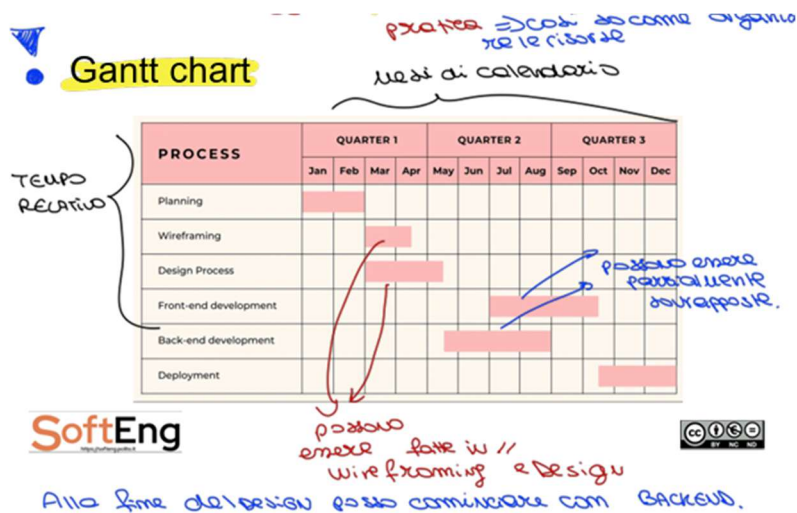
WBS (Work breakdown structure) : **scomposizione gerarchica del lavoro** necessario per completare il progetto. Si prende l'obiettivo finale e lo si divide in attività sempre più piccole, fino ad arrivare ai **work package**, cioè pacchetti di lavoro abbastanza piccoli da poter essere assegnati a una persona o a un team. Risponde alla domanda: "Che cosa dobbiamo fare", quindi non descrive il prodotto ma tutte le attività necessarie per realizzarlo. Si parte dal progetto e lo si divide in fasi o deliverable (analisi, design, test...). Ogni fase viene divisa in attività assegnate a una o + persone (work package).

PBS (Product Breakdown Structure): Non si divide il lavoro ma il prodotto finale. Che cosa dobbiamo consegnare ? Elenca tutti i deliverable

Tipicamente si usano insieme, prima si costruisce la PBS (per capire esattamente cosa bisogna consegnare, TUTTI I DELIVERABLE) e poi la WBS (per capire quali attività servono a costruire ciascun componente della PBS).

Una volta costruita la WBS sappiamo quali attività esistono e quanto lavoro c'è, bisogna sapere quando svolgere le varie attività quindi si costruisce **il DIAGRAMMA**

DI GANTT. Prende tutte le attività della WBS e le dispone lungo una linea temporale. Alcune attività possono procedere in parallelo. Permette di vedere la durata complessiva del progetto (è uno strumento di pianificazione temporale), quando termina o inizia un'attività, quali attività sono parallele e quali dipendono dalle altre.



Il **Critical Path Method (CPM)** è una tecnica di pianificazione che serve a individuare il **percorso critico**, cioè la **sequenza di attività dipendenti che richiede più tempo** per essere completata. Prendiamo le attività (percorso) con + dipendenze. Questo percorso determina la **durata minima dell'intero progetto**. Se anche una sola delle attività critiche ritarda, **ritarda automaticamente tutto il progetto**.

Il **COCOMO II (Constructive Cost Model II)** è un modello utilizzato per stimare il tempo e lo sforzo necessari per sviluppare un software.

Una delle formule principali è:

$$TDEV = 3 \times (PM)^{0.33+0.2(B-1.01)}$$

dove:

- TDEV = tempo di sviluppo stimato (mesi di calendario);
- PM (Person-Months) = sforzo richiesto;
- B = fattore che tiene conto della complessità del progetto.

L'idea è semplice:

dato uno sforzo (PM), COCOMO stima quanti mesi serviranno per completare il progetto.

Il parametro **B** dipende da **5 Scale Factors**, cioè cinque caratteristiche del progetto che influenzano la difficoltà dello sviluppo.

Fattore	Significato
PREC (Precedentedness)	Esperienza dell'organizzazione in progetti simili. PREC alto = tanta esperienza quindi il progetto risulta più semplice (ha già fatto progetti simili).

FLEX (Development Flexibility)	Quanto il processo può essere modificato, quanto è flessibile. Startup hanno flex alto perché se il cliente cambia idea, possono cambiare velocemente architettura, tecnologie.... Un sw per una centrale nucleare invece ha flex basso perché ogni requisito è imposto dalla normativa.
RESL (Risk Resolution)	Quanto bene sono stati analizzati e ridotti i rischi. Se analizziamo tutti i rischi prima, si conoscono già molti problemi, quindi il rischio è basso. Se si parte subito a programmare senza nessuna analisi, il rischio è alto.
TEAM (Team Cohesion)	Quanto il team lavora bene insieme, quanto collaborano. Non giudica le competenze dei lavoratori.
PMAT (Process Maturity)	Quanto l'azienda è organizzata per fare attività di questo tipo: ha documentazione, controllo qualità, procedure standard → tutti sanno come lavorare. Se ogni progetto viene gestito in modo diverso e ogni sviluppatore lavora come preferisce → pmat basso.

Ogni fattore viene valutato da: Very Low (VL), Low (L), Nominal (N), High (H), Very High (VH). A ogni livello corrisponde un valore numerico che contribuisce al calcolo di **B**.

GESTIONE DEL RISCHIO

Il **Risk Management** è il processo con cui si individuano, si analizzano e si gestiscono i rischi che potrebbero compromettere il progetto. Un **rischio** è: **un evento futuro che potrebbe avere un impatto negativo sul progetto.**

Esistono due approcci.

- **Reactive Risk Management** → Si interviene **solo quando il problema si presenta**. Si attuano delle correzioni per reagire al problema. È un approccio correttivo.
- **Proactive Risk Management** → Si cerca invece di **prevenire il rischio** prima che si manifesti. Approccio preferibile, cerco di evitare il problema alla radice.

3 categorie di rischio:

Project Risks → rischi legati alla gestione del progetto. Es: stime di budget errate; ritardi; mancanza di personale; risorse insufficienti;

Technical Risks → o rischi legati alla realizzazione tecnica del software. Ad esempio: tecnologia non adatta; se mi chiedono una cosa troppo complicate da realizzare

Business Risks → riguardano il mercato e il business. Se il mercato cambia, non c'è mercato per quell'app e quindi non è vendibile.

- RISK IMPACT: loss associata ad un evento
- RISK PROBABILITY: probabilità che un evento si verificherà
- RISK CONTROL: quanto posso controllare il rischio (grado con cui posso cambiare outcome)
- RISK EXPOSURE: Risk probability × risk impact

TIPI DI RISCHIO:

Tipo di rischio	Descrizione
Known Risks (Rischi noti)	Rischi già conosciuti e facilmente identificabili prima dell'inizio del progetto. Es: requisiti poco chiari.
Predictable Risks (Rischi prevedibili)	Rischi che possono essere previsti grazie all'esperienza maturata in progetti simili.
Unknown Risks (Rischi sconosciuti)	Rischi completamente imprevedibili che non possono essere identificati in anticipo.

RISK MANAGEMENT PROCESS:

Fase	Obiettivo
1. Risk Identification	Individuare tutti i possibili rischi del progetto (eventi che possono creare problemi). Prima bisogna sapere quali rischi esistono.
2. Risk Analysis	Per ogni rischio si stima Probability, Impact ed Exposure . E' una valutazione quantitativa: Exposure = Probability × Impact.
3. Risk Ranking	Stabilire quali rischi affrontare per primi (dal + pericoloso al meno pericoloso). Si interviene prima sui rischi con maggiore exposure.
4. Risk Planning	Pianificare come gestire i rischi più importanti. Devo capire se il rischio è accettabile o no. Si scelgono le strategie di mitigazione. Posso evitare il rischio (cambiare i requisiti), trasferirlo (compro assicurazione), o accettarlo e faccio delle azioni correttive
5. Risk Monitoring	Controllare continuamente i rischi durante il progetto. Il monitoraggio dura fino alla fine del progetto. Si fa un risk log.

LAVORO IN TEAM

TOP → team da 4 persone

BIG FIVE: definisce la personalità attraverso 5 tratti

Fattore	Misura
Neuroticism	Stabilità emotiva / gestione dello stress
Openness	Curiosità, creatività, apertura mentale
Conscientiousness	Organizzazione, responsabilità, perseveranza
Extraversion	Socievolezza, comunicazione
Agreeableness	Gentilezza, collaborazione, empatia

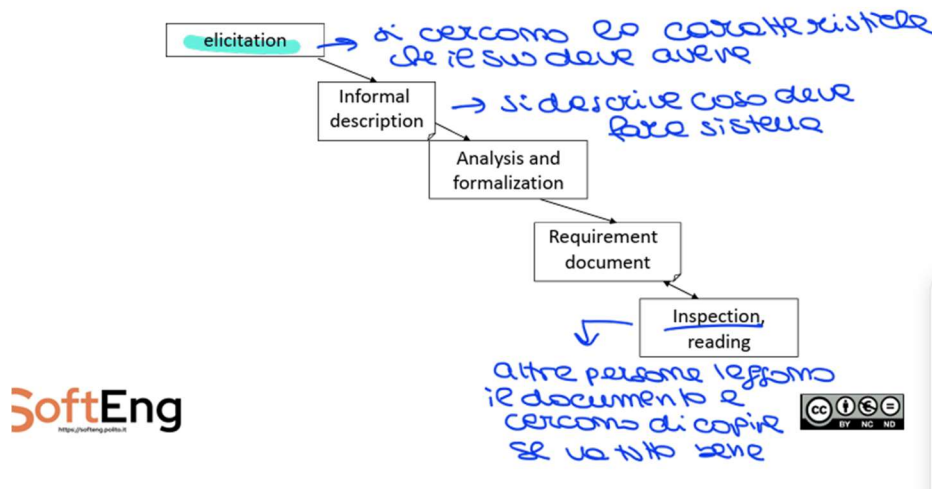
MAYERS-BRIGGS → 4 dimensioni (16 tipi di personalità):

Dimensione	Significato
E – I	Estroversione ↔ Introversione
S – N	Sensazione ↔ Intuizione
T – F	Pensiero ↔ Sentimento
J – P	Giudizio ↔ Percezione

REQUIREMENT ENGINEERING

= definire le proprietà del prodotto prima di svilupparlo, descrivendo cosa il sistema deve fare e producendo il **Requirements Document**. Una corretta definizione dei requisiti è fondamentale perché gli errori introdotti in questa fase sono i più costosi da correggere. Senza questo non sappiamo cosa testare.

Requirement engineering



Un **requirement** è una **descrizione delle proprietà desiderate del prodotto**, cioè una richiesta su ciò che il sistema dovrebbe fare o su come dovrebbe comportarsi.

Una **product property** è invece una **caratteristica effettivamente posseduta dal software realizzato**. Quindi i requisiti definiscono le proprietà desiderate; il prodotto finale può soddisfarle oppure no.

Un buon documento dei requisiti deve essere:

- **Complete**, cioè contenere tutti i requisiti necessari.
- **Consistent**, cioè privo di contraddizioni tra i requisiti.

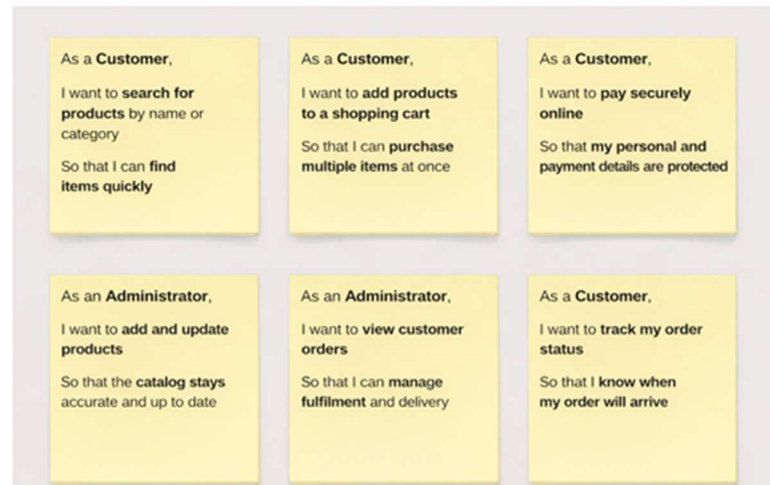
Inoltre deve evitare difetti (defects) come: omissioni; ambiguità; inconsistenze; informazioni errate; informazioni inutili; ridondanze. Difficile da ottenere in progetti grandi.

- 1) **REQUIREMENTS ELICITATION (RACCOLTA DEI REQUISITI)**: Il processo con cui si scopre **cosa vuole realmente il cliente** e cosa dovrà fare il sistema. L'obiettivo è raccogliere i requisiti parlando con gli stakeholder (cioè tutte le persone interessate al progetto: clienti, utenti, manager, manutentori, ecc.). Ci sono diverse tecniche: interviste, questionari, ethnography (osservazione diretta degli utenti mentre lavorano), focus group (discussione con stakeholder, idee, opinioni). Se il requisito non è preciso, ogni sviluppatore potrebbe implementare qualcosa di diverso (requisiti informali).

Alla fine viene prodotto il **Requirements Document**, cioè il documento ufficiale dei requisiti. E' composto da:

- **OVERALL DESCRIPTION**: informazioni generali (obiettivo, scopo, utile per capire di cosa parla il progetto, senza dettagli tecnici, alto livello).
- **STAKEHOLDERS**: qui vengono elencate tutte le persone interessate al progetto (non necessariamente ha un account nel sistema). Si parla di **STORIES AND PERSONAS**: per definire i requisiti dobbiamo profilare le persone. Vengono proprio inventati dei personaggi e gli viene dato un contesto per capire meglio. In base al profilo dettagliamo le caratteristiche di applicazione. Si parla di "profiling". ES: sconto per primo visitatore del sito, proporre prodotti simili ad un eventuale consumatore.

User Stories: Example

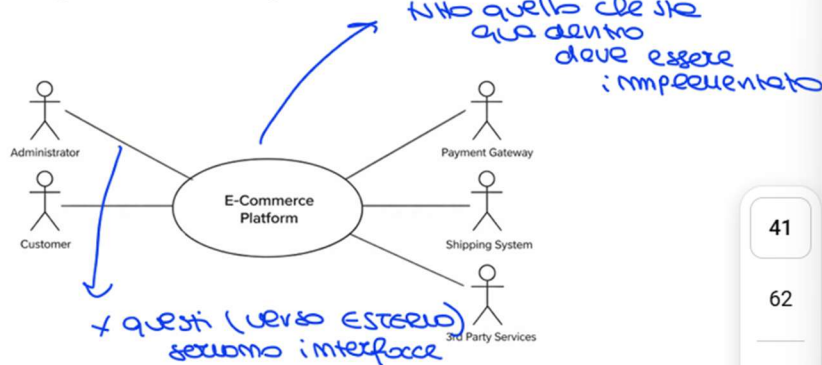


- **CONTEXT DIAGRAM AND INTERFACES:** che cosa fa il sistema e con chi comunica. Traccia la linea tra quello che è DENTRO O FUORI. Mostra cosa sta dentro o è esterno al sistema. Tutto quello che è dentro lo devo implementare, per tutto quello che è esterno invece devo implementare un'INTERFACCIA (API, logiche, fisiche (collegate a internet), grafica per user). Definisce i boundary del sistema.

Il **contenxt diagram** definisce il confine del sistema, identificando gli attori esterni e tutti i sistemi con cui il sw interagisce. Le interfacce invece descrivono come avviene questa comunicazione, specificando gli input, gli output e le interazioni tra il sistema e l'ambiente esterno.

da USE CASE DIAGRAM

Context diagram: Example



41

62

^

Interfaces – tabular form

Actor	Physical Interface	Logical Interface
Customer (Shopper)	Web browser or mobile app + Internet	Web Graphical User Interface (browse/search, product page, cart, checkout, account); optionally public/mobile APIs
Administrator / Backoffice staff	Web browser + Internet (often via VPN/corporate network)	Admin Web GUI (catalog management, pricing/promotions, orders/returns); optionally operational CLI/console for maintenance
Payment service (PSP / Card gateway)	Internet connection (TLS)	Payment APIs (authorize, capture, refund), 3DS redirect/challenge flow, status webhooks/callbacks, idempotency and error handling contract
Inventory & Catalogue system (ERP/PIM)	Internet or private network; sometimes VPN/SFTP	APIs (REST/GraphQL) or batch feeds (CSV/EDI) for products/prices/stock; synchronization rules and update events

Tipo di interfaccia	Descrizione	Esempio
1. User Interfaces (GUI)	Definiscono come l'utente interagisce con il sistema. Descrivono schermate, pulsanti, menu, campi di input, ecc.	Pagina di login, schermata di registrazione, carrello di un e-commerce. INTERFACCIA GRAFICA
2. Procedural Interfaces	Definiscono come due componenti software comunicano tramite procedure, funzioni o API. Specificano quali operazioni possono essere chiamate, con quali parametri e cosa restituiscono.	API login(username, password) che restituisce un token di autenticazione.
3. Data Exchanged	Definiscono quali dati vengono scambiati tra sistemi, il loro formato e la loro struttura.	Un servizio invia un file JSON contenente nome, email e data di nascita dell'utente.

Context diagram mi permette di capire lo SCOPO del sistema, cosa è dentro (cosa dobbiamo implementare), cosa è fuori (cosa stiamo usando, es mappe).

- REQUIREMENTS: definisce cosa deve soddisfare il sistema. Può essere: functional / non functional / domain (particolare requisito non funzionale che dipende dal contesto in cui opera il sw, dominio applicativo).
- USE CASE DIAGRAM: rappresentazione grafica delle funzionalità, serve per capire come l'utente interagisce con il sistema.

- SCENARIOS: insieme di casi d'uso, come gestire le problematiche, descrivono passo passo come viene eseguito un CASO D'USO. Descrive le sequenze di interazioni.
- GLOSSARIO: toglie ambiguità. Come si colloca il termine tecnico nel contesto del progetto (es. Vendita). Ci racconta tutti i dettagli + info tecnica.
- SYSTEM DESIGN: panoramica di tutti i componenti e di come sono connessi.

Possono essere:

- FUNZIONALI → cosa deve fare il programma (vincoli operativi)

- Numbering can be hierarchical if multiple requirements are related

Requirement ID	Description
F1	Handle online purchase transaction
F1.1	Validate shopping cart contents
F1.2	Process payment transaction
F1.3	Confirm order and generate receipt
F6	Manage shopping cart
F6.1	Add product to shopping cart
F6.2	Update product quantity in cart

spiegano meglio F1

oftEna



- NON FUNZIONALI (ISO 9126) → affidabilità/ disponibilità (quanti bugs sono stati segnalati in un periodo temporale o per che % di tempo non è disponibile il prodotto, es aggiornamenti) + usabilità (sforzo richiesto per imparare come usare il prodotto, se prodotto difficile da usare, no good) + performance + mantenibilità (sforzo, inteso come person hours, richiesto per raggiungere, modificare o cancellare la funzione di un sw, o sforzo per rimediare un difetto) + efficienza (response time in termini di risorse necessarie) + portabilità (sforzo per rimpiangere l'app in un'altra piattaforma) + funzionalità (capacità di eseguire le funzionalità previste in TUTTI I CASI) + SECURITY AND SAFETY (ISO 25010): la security sono i privilegi, ossia fornire info solo ad un utente autorizzato, safety è assenza di problemi di sicurezza delle persone, no danni alle persone. Se ci sono problemi qui il progetto fallisce. Vincoli entro cui devo operare.

CASI PARTICOLARI DI REQUISITI NON FUNZIONALI:

- Organizational: dipendono dal CONTESTO in cui sviluppo il sw e non dal sw stesso.
- Domain: dipendono dal PROCESSO, da fattori esterni al sistema ma propri del processo di sviluppo.

Non-functional requirements: example

Attenzione a dati soggetti che portano ad AMBIGUITA'.

Requirement ID	Description
NF1(efficiency)	Function F1.1 less than 1msec
NF2 (efficiency)	Each function less than ½ sec
Domain1	Currency is Euro – VAT is computed as ..

I requisiti devono essere MISURABILI! Ci possono essere conflitti tra requisiti non funzionali in sistemi complessi (es efficienza e sicurezza).

MAIN PURPOSE OF REQUIREMENTS ENGINEERING: definire le product properties prima che lo sviluppo inizi.

Avere dei requisiti non funzionali non misurabili comporta problemi perché sono difficili o impossibili da verificare oggettivamente.

USE CASES

E' un metodo complementare di requirements elicitation → come interagisce utente con sistema. **Mostra ATTORI (utenti e sistemi) + FUNZIONALITA' SISTEMA.**

SCENARIOS:

Uno **scenario** è una sequenza concreta di passi che descrive **come un utente interagisce con il sistema** in un caso specifico. Le operazioni devono essere fatte in un certo ordine , il TEMPO è fondamentale qui: l'ordine temporale conta, è una storia ordinata nel tempo. Mentre un requisito funzionale dice COSA DEVE FARE IL SISTEMA, lo scenario dice COME AVVIENE CONCRETAMENTE QUELLA COSA (es effettuare un ordine) PASSO PASSO.

- PRECONDITION: vincoli che devono essere soddisfatti per fare una certa cosa. Es: prodotto deve esistere nel catalogo affinché possa essere scelto, o autenticazione utente.
- POSTCONDITION: condizione da soddisfare alla fine dello scenario: es pagamento registrato, ordine creato nel sistema.

ESEMPIO: SCENARIO DI LOGIN:

1. il cliente apre la pagina di login;
2. il sistema mostra il form;
3. il cliente inserisce username e password;
4. il sistema valida le credenziali;

5. il sistema autentica il cliente.

Qui la precondition è: Customer is not authorized. Cioè il cliente non è ancora autorizzato/loggato.

La postcondition è: Customer is authorized and authenticated. Cioè, alla fine, il cliente è riconosciuto dal sistema e può accedere.

USE CASE: insieme di scenari che hanno un obiettivo comune. ESEMPIO: devo fare un ordine (obiettivo comune), posso farlo con diversi scenari: pagamento con carta, pagamento fallito, ordine annullato, rimozione prodotto da carrello.

Un caso d'uso descrive un insieme di interazioni orientate a un obiettivo tra attori esterni e sistema. Ha un tempo limitato, uno o + utenti, un obiettivo comune, tempo e luogo ben definiti.

Elemento	Significato
Actor	Chi usa o interagisce con il sistema
System	Il sistema considerato come black box (context diagram)
Goal	L'obiettivo che l'attore vuole raggiungere

ESEMPIO:

Elemento	Esempio
Actor	Customer
System	E-commerce
Goal	Comprare un prodotto

USE CASE DIAGRAM

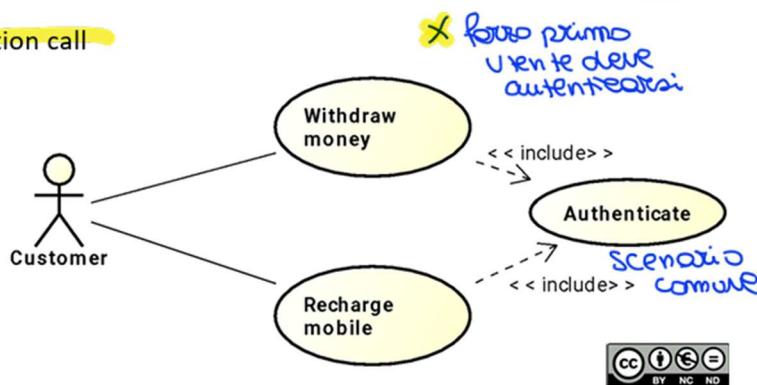
= diagramma UML che rappresenta graficamente gli use case. Mostra gli attori, casi d'uso, relazioni tra attori e casi d'uso + confini del sistema. Mostra solo una visione riassuntiva e non contiene tutti i dettagli. La narrativa dettagliata è data dagli scenarios.



Gli attori possono essere: PRIMARI (protagonista, chi interagisce di più con il sistema), o secondari (passivi, viene interrogato per fare solamente certe cose). L'attore è qualcosa di esterno al sistema che interagisce con esso, può essere una persona, un altro sistema, un dispositivo hw. L'attore NON fa parte del sistema ma comunica con esso. Gli attori sono un sottoinsieme degli STAKEHOLDERS (chiunque abbia un interesse nel sistema), invece l'attore è uno stakeholder che interagisce direttamente con il sistema.

INCLUSIONE:

tion call



E' una sottofunzione obbligatoria.

GENERALIZATION:

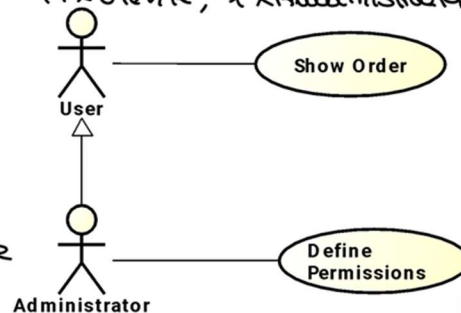
Generalization

- A generalization from an actor B (Administrator) to an actor A (User) indicates that an instance of B can interact with the same use-case as an instance of A (not vice versa)

Sia utente che amministratore in questo caso possono fare "Show Order". Solo amministratore può definire i permessi.

SoftEng
http://softeng.polito.it

Non dobbiamo fare 2 account (1x utente, 1x amministratore)

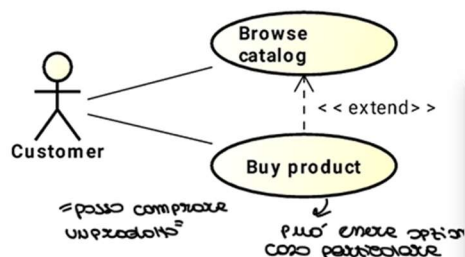


EXTENSION:

Extension → *un caso d'uso può essere eseguito*

- A UC extend another UC when it represent a possible (though not mandatory) prosecution of the extended UC

Posso comprare un prodotto direttamente, oppure attraverso link da catalogo

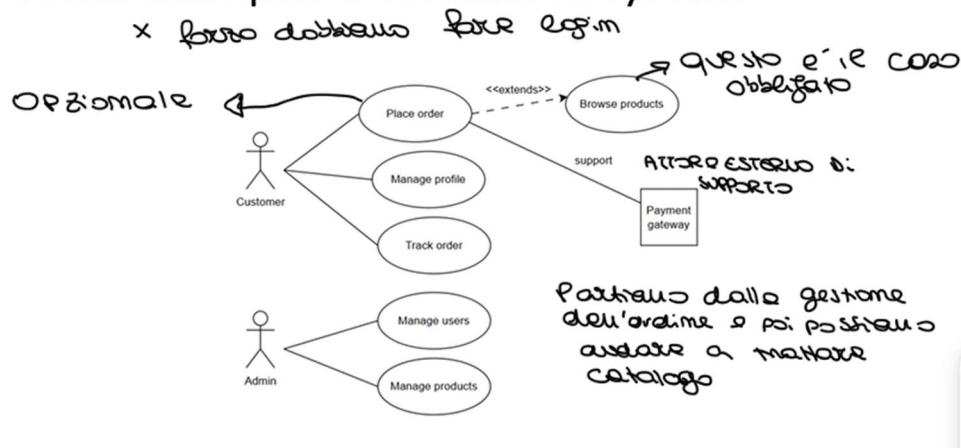


SoftEng
http://softeng.polito.it

Inclusione → mi obbligo (devo comprare un prodotto)

ESEMPIO:

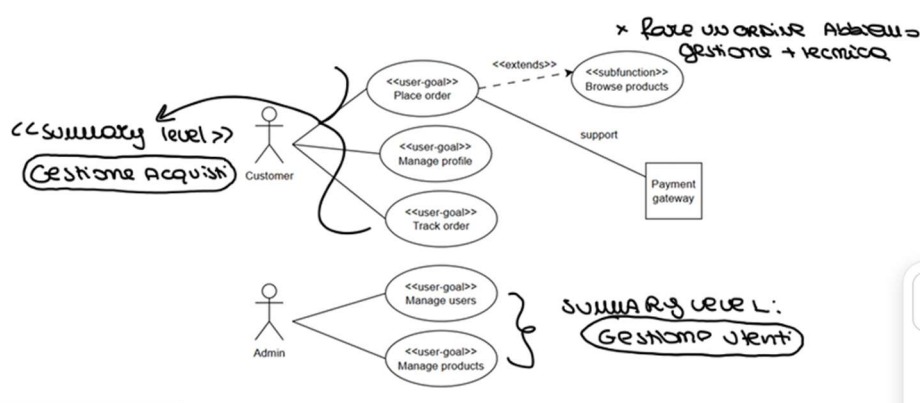
A full example: e-commerce system



3 livelli di caso d'uso:

- SUMMARY LEVEL: alto livello, + complicato. Serve a delineare le macro funzionalità. Non scende nei dettagli operativi ma raggruppa tanti user goal insieme per dare una visione + ampia dell'app. Es: transaction
- USER GOAL LEVEL: Descrivono il **goal (obiettivo) specifico** che un attore primario o un utente vuole raggiungere usando il sistema. Tanti user goal compongono un summary level. Es: prelevare contanti.
- SUBFUNCTION: livello + basso della gerarchia, da info di basso livello per ogni attività. Descrive un blocco funzionale tecnico più piccolo per far funzionare il livello superiore (es validare i dati della carta, o autenticarsi nel sistema).

Granularity of use cases



Titolo → stesso che abbiamo messo nel caso d'uso, altrimenti incoerenza !

The SWEED Template

x dettagliare quello che è nel Brief

Use case	Name of the use case = a short active verb phase stating the goal
Scope	The scope of the "system" being considered (enterprise – system – component)
Level	Summary / user-goal / <u>sub-function</u> <i>Andiamo molto in dettaglio</i>
Intention in Context	The primary action's intention and the context within which it is performed
Primary actor	The primary actor of the use case
*Support actor	The optional secondary actor

SoftEng
http://softeng.polito.it

→ OB3 principale (perché usiamo sistema)
→ potrebbe non f (metto =)



The SWEED Template

es: amministratore di condominio ha interesse che gestione conti dia in ordine.

da prima nell'input
mentare caso d'uso
(ed es se
interesse
molti stakeholder
dato lo
impegnato
prima)

*Stakeholder's interests	The list of stakeholders and their interests in the use case <i>Si vede se uno di questi ha interesse specifico.</i>
*Precondition	What is necessary in the system and its environment for the use case to be executed
*Minimum Guarantees	Which interests of the stakeholders are protected in all circumstances
*Success Guarantees	The state of the system and the environment if the goal of the primary actor succeeds
*Trigger	What event starts the use case <i>(es: sveglia)</i>

se non c'è questo (ad es Account Ologim) il caso d'uso non parte.

SoftEng
http://softeng.polito.it

solitamente utente clicca
(viene scatto quando sistema
avviato evento)

Attivato che viene scatto anche solo all'avvio del caso d'uso, ad es: assegnazione codice a domanda.



sveglia → utente

TRIGGER → qualcosa che fa partire in automatico

The SWEED Template

utente - sistema

Main success scenario	<step_number>.<action_description> The numbered steps of the scenario from the trigger to the goal delivery (and possible clean-up)
*Extension	<step_altered> <condition> : <action_description> or <sub-use_case> Description of condition and alternatives

cosa succede quando va tutto bene

step che può avvenire o no (es: autenticazione superata)

cosa succede se qualcosa va storto o ≠ da come avevo previsto

SoftEng <http://softeng.polito.it>

CC BY NC ND

Il livello di precisione qui è DETAILED (non BRIEF).

Per extensions:

- Alternate success path
- Primary actor behaves incorrectly
- Inaction by the primary actor
- Negative outcome of validation
- Internal failure (expectable)
- Unexpected or abnormal failure
- Critical performance failure

Devo sempre partire dal punto di vista del sistema che riceve a input utente.

- An extension step takes place in addition to the respective main step, notation: 2 ||, l'estensione si riferisce sempre ad uno step del main success scenario.
- or as an alternative notation: 2a

Example: Place order use case

Use Case	Place Order
Scope	E-Commerce App
Intention	Purchase selected products and complete payment.
Actor	Customer
Main Success Scenario	1. Customer searches a product. 2. System displays the list of matching products. 3. Customer selects a product. 4. System displays product details. 5. Customer adds the product to the shopping cart. 6. System updates and displays the shopping cart. 7. Customer proceeds to checkout. 8. System displays the order summary and asks for shipping information. 9. Customer enters shipping information. 10. System displays available shipping and payment options. 11. Customer selects shipping and payment method and confirms the order. 12. System processes payment, records the order, and displays order confirmation.

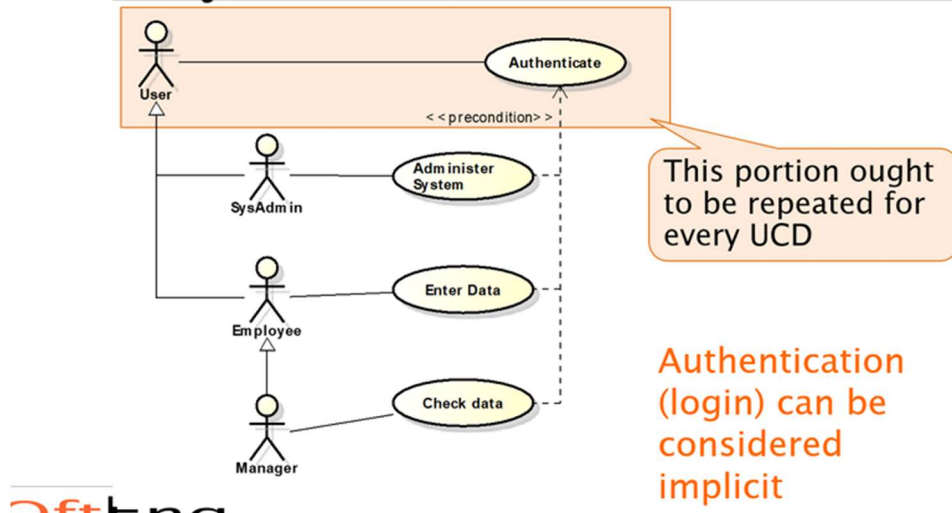
Example: Place order use case

Extensions	5.a Product becomes unavailable: 5.a.1 System informs the customer that the product is out of stock 5.a.2 Customer selects another product 9.a Shipping address invalid: 9.a.1 System asks customer to correct the address 9.a.2 Customer enters a valid address 11.a Payment rejected: 11.a.1 System informs the customer that payment failed 11.a.2 Customer selects another payment method 7.a Customer cancels the purchase: 7.a.1 System returns to product browsing
------------	--

Ad es si preferisce riportare il login come PRECONDITION piuttosto che come INCLUDE.

Authentication

Utente generale → Qui si autentica

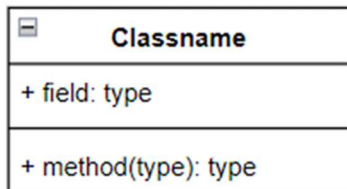


GLOSSARIO / CLASS DIAGRAM

Ci permette di rappresentare: classi, istanze/oggetti, attributi di alto livello, associazioni (no implementazione).

CLASSI: un insieme di oggetti con proprietà comuni + esistenza autonoma.

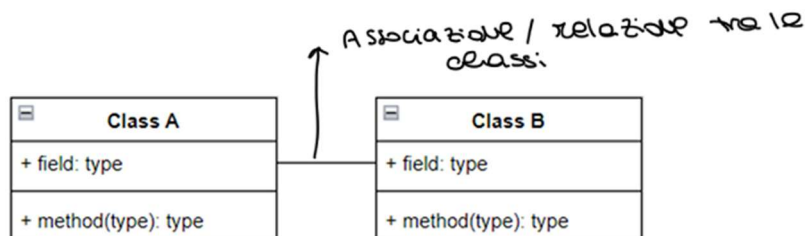
Attribute types



Type	Description
Int	Integer number
Float	Real number
Boolean	Logical value (true/false, yes/no)
String	Character string / text
Date	Date (day-month-year)
Time	Time (hours:minutes:seconds)
Currency	Currency values (€)

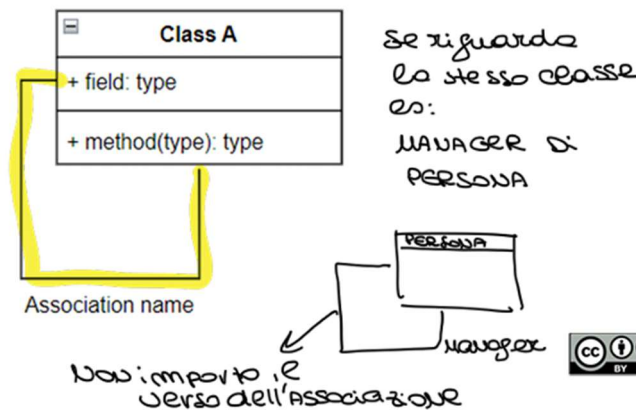
I fields sono gli ATTRIBUTI (proprietà elementari della classe).

ASSOCIAZIONI: link logici tra 2 classi:



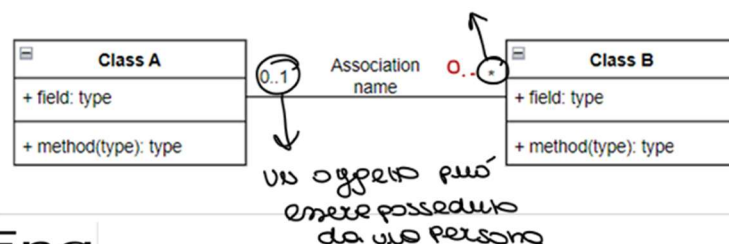
Possono essere:

- RICORSIVE: associazione tra classe e se stessa.



MOLTEPLICITA':

In questo caso sono entrambi i versi obbligatori



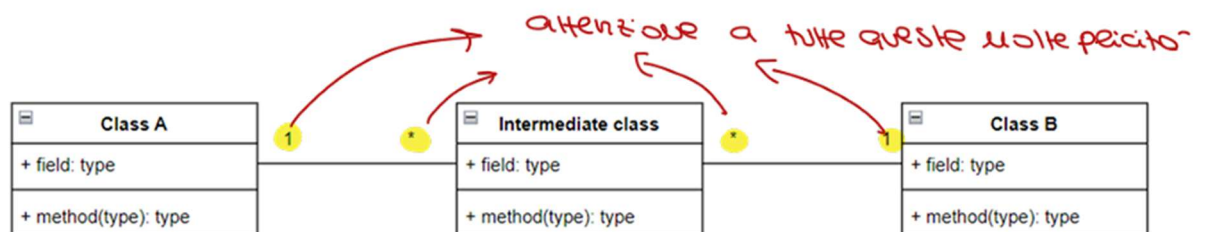
Molteplicità 0 → opzionale.

Multiplicity → si scrive sempre nella classe di DESTINAZIONE.

n	Exactly n
$*$	Zero or more
$m..n$	Between m and n (m,n included)
$m..*$	From m up
$0..1$	Zero or one (optional)

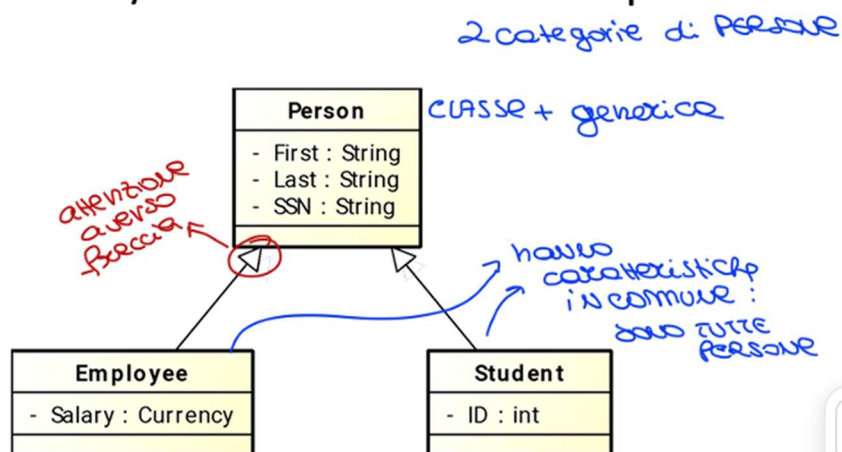
CLASSI INTERMEDIE:

Se ho una relazione molti a molti metto una classe nel mezzo:



Non mettere come attributi i DERIVED ATTRIBUTES che derivano dal calcolo di una molteplicità ad esempio perché si possono dedurre e sono dati instabili.

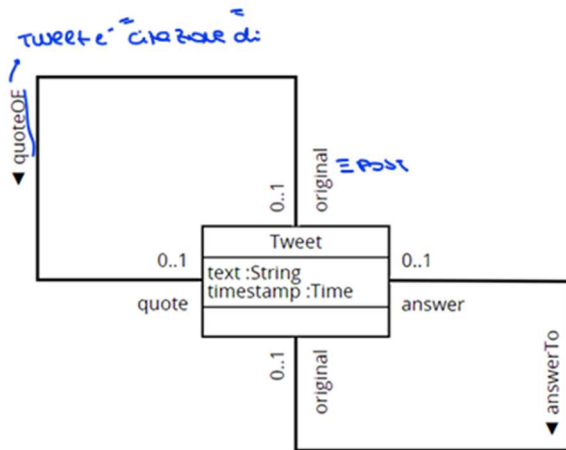
Specialization / Generalization: Example



OftEnd

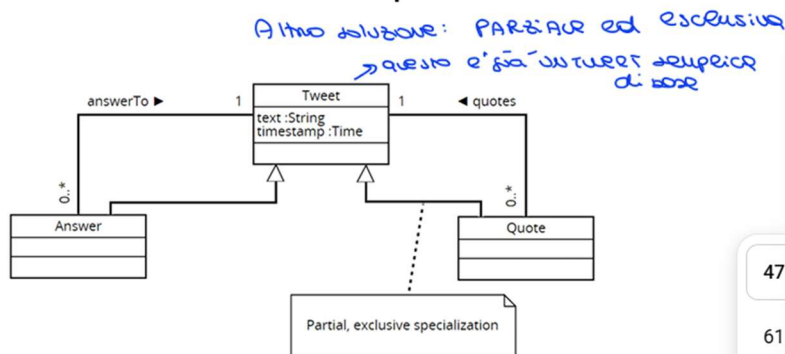


ESEMPIO PER COMMENTI, RISPOSTE A POST:



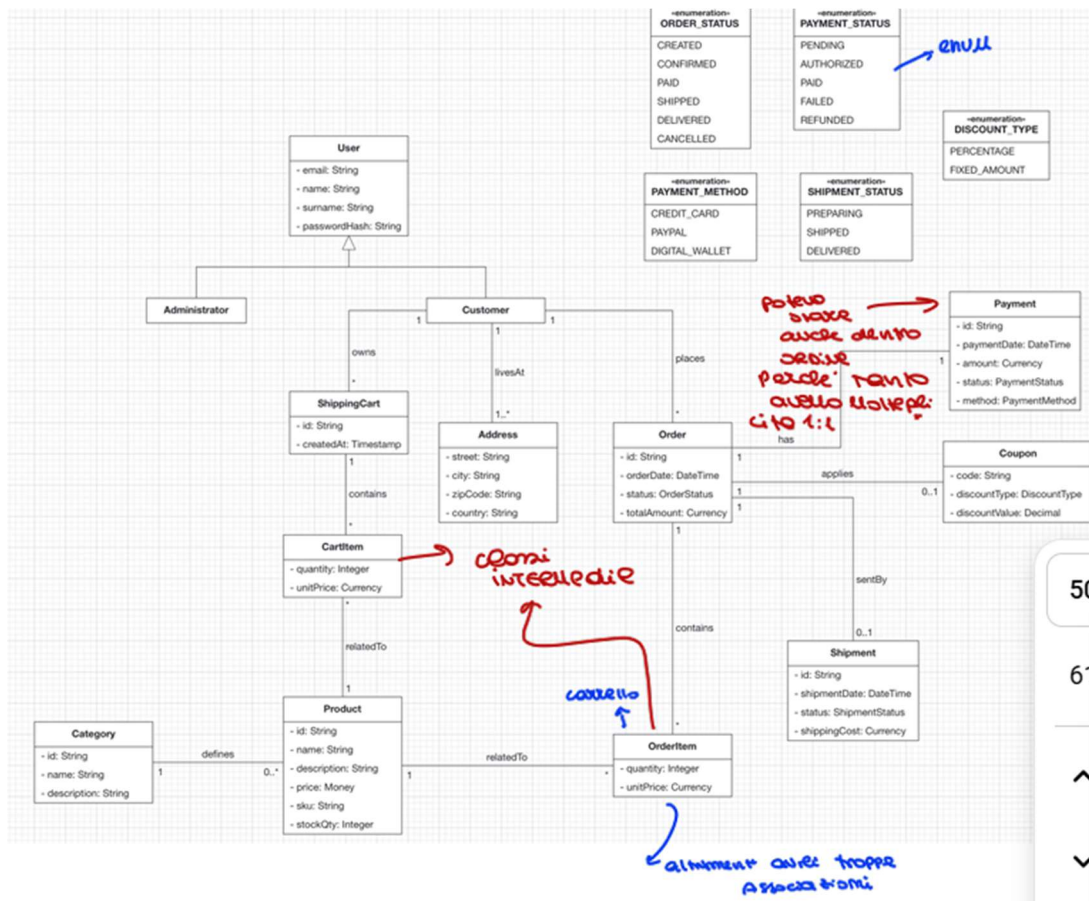
Meglio fare come sotto:

Example: Tweets – Partial Specialization



47

61



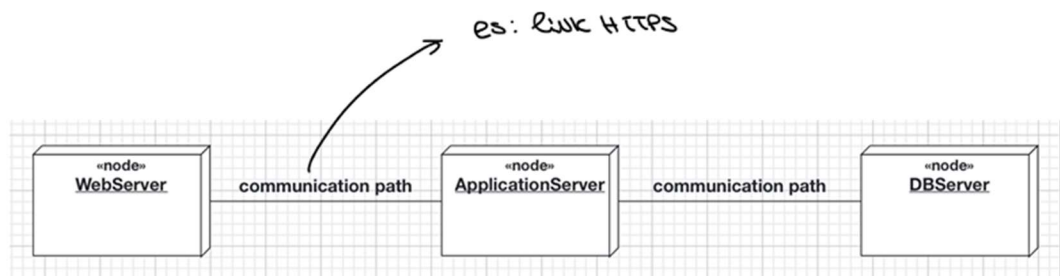
50

61

DEPLOYMENT DIAGRAM (per system design)

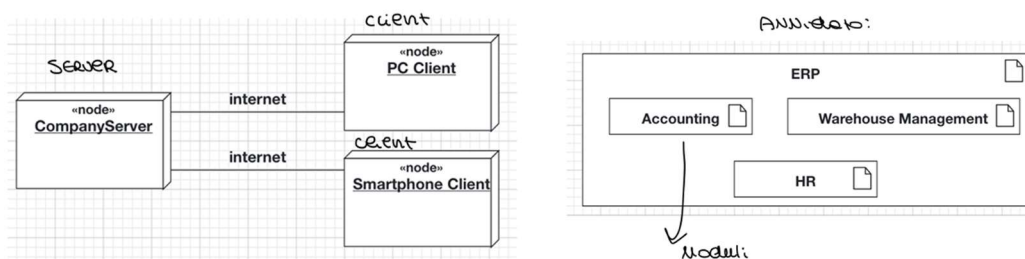
Rappresenta **come il software viene distribuito sull'hardware**, mostrando i **nodi fisici** (server, PC, smartphone, database, ecc.) e le **connessioni** tra di essi. **In breve per l'esame:** descrive **dove vengono eseguiti i componenti software** e **come comunicano tra loro attraverso la rete**.

NODO: entità fisica / sw che ci da un posto dove abbiamo delle capacità computazionali.



Example

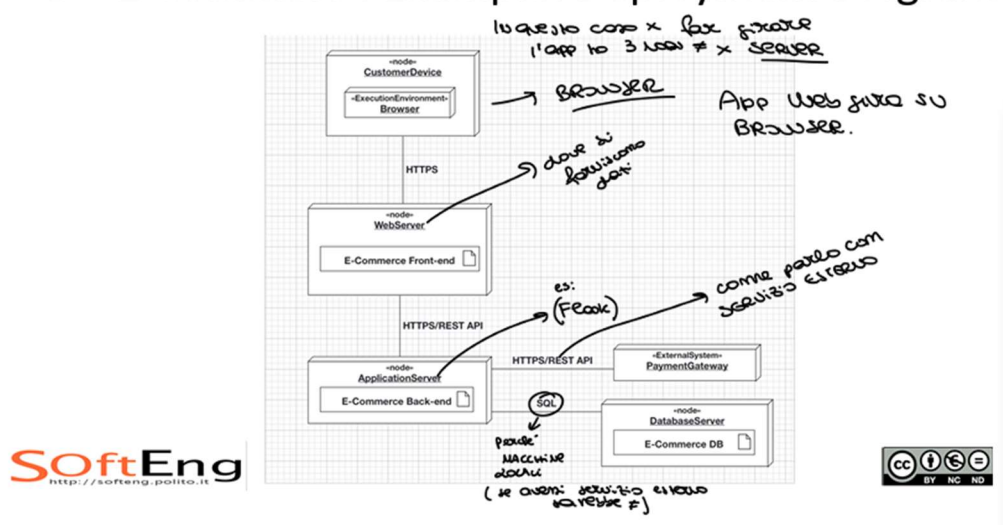
Qui: non draw alcun deployment



Quelli interni sono gli ARTIFACT: prodotto sw (libreria, file eseguibili, librerie, componenti nei microservizi...).

Come deve essere implementato il sistema

8 – E-Commerce Example: Deployment Diagram



VALIDATION- VERIFICATION

Mano a mano che andiamo avanti con il nostro programma aumenta il costo. Il costo è maggiore per fixare i bugs. GOALS: ridurre al minimo il numero di defects (impossibile arrivare a 0), massimizzare il numero di defects scoperti e rimossi, minimizzare tempo tra insertion-discover-removal. I defects relativi ai requisiti sono i più costosi. All'inizio non sappiamo bene dove vogliamo arrivare con la nostra app quindi i bugs sono maggiori. Più tempo un defect rimane nel sistema e + costa rimuoverlo.

BUGS:

- FAILURE: evento in esecuzione in cui il sw si comporta in un modo inaspettato
- FAULT: feature del sw che causa una failure (errore nel codice, requirement incorretto).
- DEFECT: failure o fault. E' caratterizzato da tempo in cui INSERISCO (es se sbaglio a scrivere la funzione)- DISCOVER – REMOVAL (tempo in cui rimuovo).

Il **technical debt** è il “debito tecnico”: succede quando, per consegnare più velocemente, si rimandano attività di qualità come testing, refactoring, design pulito o documentazione. È come un debito economico: oggi risparmio tempo, ma domani dovrò “ripagare” con interessi. Es: faccio copia-incolla invece di creare una funzione comune: oggi finisco prima, ma se c'è un bug devo correggerlo in tutti i punti duplicati.

2 tecniche:

Tipo	Significato	Esempi
Statiche	Analizzano codice/documenti senza eseguire il programma. NON ESEGUO CODICE	inspection, code review, static analysis
Dinamiche	Controllano il software eseguendolo. ESEGUO CODICE	testing

Per i requisiti: ispezione del documento dei requisiti (statico), product prototype (dinamico).

Per design: ispezione del documento di design (statico)

Per implementazione: testing (dinamico) code analysis (statico).

CODE INSPECTION: (ora la fa IA), review del codice in modo sistematico (non eseguo niente, leggo il codice e basta). Non è testing perché il programma non viene lanciato, ma serve a trovare i defects presto, quando costano meno da correggere (GOAL). Es: codice difficile da leggere, requisiti ambigui, duplicazione.... Le inspection sono ancora utili perché gli strumenti automatici trovano molti errori tecnici, ma spesso non capiscono bene il significato del codice o l'intenzione del requisito (invece le inspections: individuano i problemi semantici, le inconsistenze nei requisiti e nel design, migliorano la comprensione condivisa all'interno del team, sono utili per requisiti, design, critical code sections). Aiutano il testing, non lo sostituiscono.

Di solito nelle aziende c'era il cosiddetto Fegan's inspection process:

Fase	Cosa succede
------	--------------

Planning	Si decide cosa ispezionare, chi partecipa e si preparano i materiali
Overview	L'autore presenta l'artefatto e il contesto
Preparation	I reviewer leggono individualmente e cercano difetti
Inspection Meeting	Si discutono i difetti trovati, senza risolverli nel meeting
Rework	L'autore corregge i difetti
Follow-up	Il moderatore verifica le correzioni

Tradizionalmente come ruoli c'erano l'autore (chi scrive il codice / documento), reviewers (cercano i defects), moderator (gestisce il processo), Scribe (logga i defects e decisions). Ora si passa da una inspection formale a un controllo qualità continuo e integrato nel flusso di sviluppo, grazie a uso di git anche. Con le inspections controlliamo la correttezza del codice, leggibilità, consistenza, duplicazione, edge cases (casi limite).

Ci sono diverse tecniche moderne per fare inspections:

Tecnica	Descrizione
Checklist-based review	Si usa una checklist di domande: correttezza, leggibilità, testabilità, standard
Scenario-based review	Si controlla l'artefatto simulando scenari concreti di uso o errore. Si simula come è stato usato il sistema, come reagisce a situazioni specifiche.
Perspective-based review	Ogni reviewer analizza da un punto di vista diverso: utente, tester, designer, manutentore

ANALISI STATICA : analisi automatica del codice senza eseguirlo. Serve a trovare gli errori prima del runtime, la prima analisi statica viene fatta dal compilatore (non esegue codice). Cerca di individuare errori sintattici, vulnerabilità di sicurezza...

METRICHE DI ANALISI STATICA:

Metrica	Cosa misura	Perché è importante
Cyclomatic Complexity (CC)	Numero di percorsi indipendenti nel codice	Più è alta, più il codice è difficile da testare
LOC	Numero di linee di codice	Misura dimensione del codice
CBO (coupling between objects)	Quante classi sono collegate a una classe	Alto coupling = meno modularità e più manutenzione
DIT (depth of inheritance tree)	Profondità della gerarchia di ereditarietà	Troppa ereditarietà aumenta complessità
LCOM (lack of cohesion in methods)	Quanto i metodi di una classe sono coesi	Bassa coesione = classe progettata male
NOM (number of methods)	Numero di metodi in una classe	Troppi metodi = troppe responsabilità

Code Duplication	Quantità di codice duplicato	Aumenta manutenzione e probabilità di errori
-------------------------	------------------------------	--

TESTING: tecnica dinamica (devo eseguire il codice). **OBJ:** trovare defects. Il test ha successo se rileva dei defect. Il testing e il **DEBUG** sono 2 cose diverse: il primo vuole trovare i problemi, se qualcosa non va, il secondo risponde a domande tipo perché non funziona, trova la causa, modifica il codice per correggere e viene fatto durante lo sviluppo per verificare che quello che stiamo facendo sia in linea con quanto richiesto. Con il debug cerco di far funzionare le cose

TEST CASE: singolo test e contiene 3 elementi: name, input ed expected output.

TEST SUITE: insieme di test.

CLASSIFICAZIONE:

- **UNIT TEST:** verifichiamo che i singoli test funzionino in isolamento, testo la singola funzione (unità più piccola). Prima cosa che faccio.
- **INTEGRATION TEST:** verifico interazione tra + unità
- **SYSTEM TEST:** verifica il Sistema completo, lo faccio alla fine
- **TEST MANUALE:** test eseguito manualmente da una persona. Non è riproducibile (scelta – furba ma – costosa)

WHITE BOX TESTING: il tester conosce il codice, sa come è implementato. Studia come è fatta la funzione e vuole coprire il maggior numero possibile di rami.

BLACK BOX TESTING: non mi interessa il codice, guardo solo input e output come se fosse una scatola nera, ipotizzo il valore restituito. Non conosco il codice sorgente ma solo come si deve comportare. I test derivano dai requisiti.

REGRESSION TEST → deve essere fatto subito. Ogni volta che modifico il codice verifico che il resto continui a funzionare, per evitare di rompere funzionalità già corrette. Rieseguo lo stesso test su una versione di codice diversa, per assicurarmi che il programma non sia invalidato da una modifica.

EXHAUSTIVE TESTING → provare tutti gli input possibili. Se lo facessi proverei al 100% l'assenza di bugs ma è impossibile. Richiede troppo tempo. Cerchiamo di essere il + vicini possibili all'exhaustive testing. (il goal del testing è trovare defects, non dimostrare che il sistema sia corretto!).

DIJKSTRA → testing rivela solo la presenza di errori, mai la loro assenza. (Secondo **WEINBERG** lo sviluppatore non deve testare il suo stesso codice, ma devono farlo altri).

PARETO-ZIPF LAW: l'80% dei defects proviene dal 20% dei moduli.

COVERAGE: entità (dipende dal tipo di test) considerate sul totale. Ad es per **UNIT TEST:** devo considerare la copertura dei **METODI**, per **INTEGRATION** considero le **DIPENDENZE**, per **SYSTEM** i **REQUISITI**.

UNIT TESTING

BLACK BOX

Il tester non conosce il codice sorgente. Lo sviluppatore è la persona meno indicata per fare test, per evitare bias. Non gli interessa sapere come è scritto il programma, vede il sistema come una scatola nera, conosce solo i requisiti, gli input, output attesi e le specifiche. I test case vengono dalle specifiche non dal codice.

Tecniche:

- **RANDOM TESTING:** non è ottimale. Prendo input a caso e verifico il risultato. Facile da applicare. E' indipendente dai requisiti (testo anche senza avere analisi dettagliata del sistema, ma per avere una buona probabilità di trovare bugs dovrei eseguire moltissimi test, per questo non è ottimale perché richiederebbe molto tempo).
- **EQUIVALENCE CLASS PARTITIONING:** dato che molti input fanno comportare il programma allo stesso modo, non ha senso provarli tutti, quindi li raggruppo in **CLASSI DI EQUIVALENZA** (= insieme di input che il sistema tratta nello stesso modo, possono essere valide / non valide). Divido l'insieme degli input in gruppi che producono lo stesso comportamento. In questo modo riduco enormemente il numero di test. Oltre a scegliere un valore rappresentativo della classe, bisogna testare i valori al confine (casi limite, boundary conditions).
CRITERI: attributo input che voglio analizzare (es: età)
PREDICATO: valore, regola (es. Età>28). Divide automaticamente gli input in classi.

Ogni classe di equivalenza deve essere rappresentata da almeno un test case. Devo avere:

- 1 test case per ogni valid input class
- 1 test case per ogni invalid input class
- Test case per boundary conditions

ESEMPIO 1:

def square_root(x: float) -> float:

criterio → x

boundary → zero, +inf e -inf

predicato → $x > 0$, individuo 2 partizioni:



ESEMPIO 2:

```
def convert(s: str) -> float:
    converts a sequence of chars (max 6) into an integer number.
    Negative numbers start with a '-'
```

Criterion to define the class	Equivalence classes and test cases	
① String represents a well formed integer	Yes T1("123"; 123)	No T2("1d3"; error)
② Sign of number	Positive T1("123"; 123)	Negative T3("-123"; -123)
③ Number of characters	≤ 6 T1("123"; 123)	> 6 T4("1234567"; err)

Handwritten notes:
 - Above the table: "mi deve dare errore" with an arrow pointing to the 'No' column.
 - Next to the table: "CRITERIO: stringa rappresenta intero"
 - Below the table: "no errore" under the 'Yes' column and "errore" under the 'No' column.
 - Bottom left: "SoftEng" logo.
 - Bottom right: "14" and "63".

facciamo tutte le possibili combinazioni delle eq. class:

Equiv. classes- combinatorial (Abbiamo 2 predicati)

WF integer	sign	N char	Test Case
yes	Pos	<=6	T1("123"; 123)
		>6	T4("1234567"; err)
	Neg	<=6	T3("-123"; -123)
		>6	T5("-123456"; err)
no	Pos	<=6	T2("1d3"; err)
		>6	T6("1sed345"; err)
	Neg	<=6	T7("-1ed"; err)
		>6	T8("-1ed234"; err)

SoftEng



Devo ingenerare anche i TEST BOUNDARY

Boundary - combinatorial

WF integer	sign	N char	Boundary Value
yes	Pos	<=6	"0" "999999"
		>6	"1000000" "9999999"
	Neg	<=6	"-0" "-99999"
		>6	"-999999"
no	Pos	<=6	" "
		>6	" " (7 blanks)
	Neg	<=6	"_"
		>6	"_"

Per i sistemi stateful, devo tenere conto anche non solo degli input, ma dello stato interno del sistema che evolve continuamente. Ad esempio se ho un sistema che prende in input una password, se l'input è "wrong password", posso ottenere output diversi ogni volta a seconda dell'input diverso: first wrong password → normal error, multiple failures → account bloccato.

Test Case	Password Correct	Current State	Expected Outcome
T1	True	Active	Login successful
T2	False	Active	Error message
T3	False	Active (2 failures)	Error + account locked
T4	True	Locked	Login denied
T5	False	Locked	Login denied

White box testing

Il tester conosce completamente l'implementazione del programma. Il codice sorgente diventa la guida per progettare i test, è una sorta di glass box (posso vedere dentro). L'obj non è solamente trovare bug, ma misurare quanto del codice è stato veramente eseguito, per questo si parla di COVERAGE:

- Statement coverage → eseguire tutte le istruzioni del programma almeno una volta. **STATEMENT COVERAGE** = #covered statements / #statements.
ESEMPIO: in questo caso servono 2 test per avere 100% coverage T1(1,1), T2(-1,1):

```
double abs(double x){
  if (x>=0) then return x;
  else return -x;
}
```

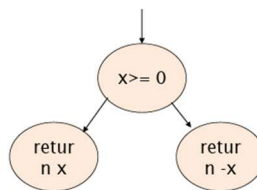
statements

- Node coverage: considero i nodi del grafo, è molto simile a statement coverage (in cui contano le istruzioni del programma).

```
def abs(x: float) -> float:
  if x >= 0:
    return x
  else:
    return -x
```

100% node coverage with T1(1;1) and T2(-1;1)

Statement coverage == node coverage



Basic blocks

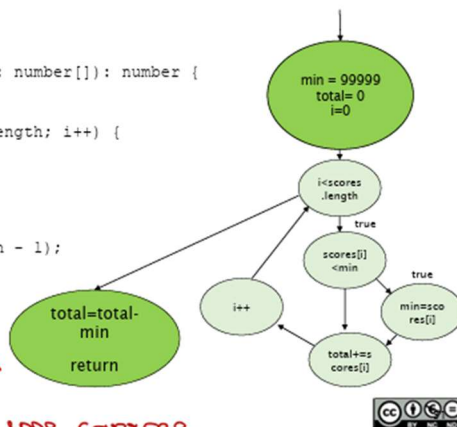
```
function homeworkAverage(scores: number[]): number {
  let min = 99999;
  let total = 0;
  for (let i = 0; i < scores.length; i++) {
    if (scores[i] < min)
      min = scores[i];
    total += scores[i];
  }
  total = total - min;
  return total / (scores.length - 1);
}
```

T1(scores[10.0,10.0] ; 10.0)

100% node coverage

SoftEng

*passando array con 2 valori
ho 100% node coverage
(con 1 solo test)*



*però questo test non ha dato
perché sto passando 2 valori =*

- Decision coverage → voglio che ogni decisione venga presa almeno una volta come True e almeno una volta come False, ogni espressione booleana deve essere valutata in entrambe le direzioni. = decisioni coperte / decisioni totali. Equivale all'edge coverage nel control graph, vuole percorrere entrambi i rami di un nodo. Nel control flow graph, l'edge coverage implica la node coverage ma non viceversa.

X poter garantire edge coverage, ho bisogno del test T1 e poi di T2 per avere 1.5

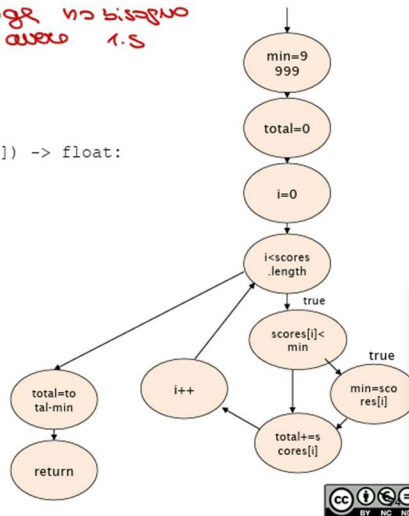
Edge coverage

```
def homework_average(scores: list[float]) -> float:
    min_val = float('inf')
    total = 0.0
    for score in scores:
        if score < min_val:
            min_val = score
        total += score
    total -= min_val
    return total / (len(scores) - 1)
```

T1({10.0}; 10.0)

T2({1,2}; 1.5)

SoftEng
https://softeng.polimi.it



- Condition coverage: una decisione può contenere più condizioni: if (A and B) or C (qui ci sono 3 condizioni: A,B,C). Condition coverage = condizioni eseguite/condizioni totali. Quindi abbiamo una concatenazione di espressioni booleane (and / or).

- Simple condition coverage: ogni condizione ha almeno un T e un F.

Simple condition coverage

```
is_married: bool
is_retired: bool
age: float
discount_rate: float
```

```
if (age > 60 and is_retired) or is_married:
    discount_rate = 30
else:
    discount_rate = 10
```

Decision coverage: vedo → .1 volta sono where
.1 volta sono where false

Test case	age>60	isRetired	isMarried	Decision
T1	T	T	T	T
T2	F	F	F	F

SoftEng
https://softeng.polimi.it



- Multiple condition coverage: faccio tutte le combinazioni possibili:

Multiple condition coverage = 100%

```
def compute_discount_rate(is_married: bool, is_retired: bool, age: float) -> float:
    if (age > 60 and is_retired) or is_married:
        return 30.0
    else:
        return 10.0
```

Faccio tutte le combinazioni possibili

Test case	age>60	isRetired	isMarried	Decision
T1	T	T	T	T
T2	T	T	F	T
T3	T	F	T	T
T4	T	F	F	F
T5	F	T	T	T
T6	F	T	F	F
T7	F	F	T	T
T8	F	F	F	F

SoftEng
https://softeng.polimi.it

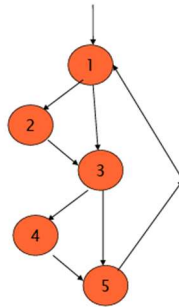


- Path coverage: un path è una sequenza di nodi in un grafo. Vogliamo testare tutti i path del programma (covered paths/total paths).

Path coverage

- Combinatorial explosion with cycle

- 1,3,5
- 1,3,5,1,3,5
- 1,3,5,1,3,5,1,3,5
- Etc ..
- $N_{paths} = 4^{n_{loops}}$



- Loop coverage: è una semplificazione del path coverage. Per ogni LOOP consideriamo 3 test cases:

- Un test case che esegue il loop 0 volte
- Un test case che esegue il loop 1 sola volta
- Un test case che esegue il loop 2 o + volte

Loop coverage

```
def homework_average(scores: list[float]) -> float:
    min_val = float('inf')
    total = 0.0

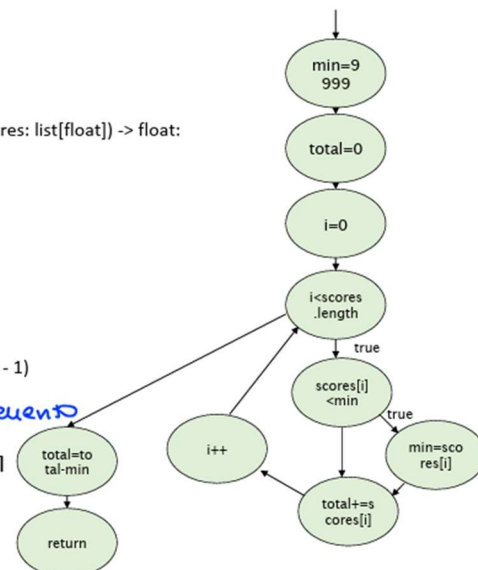
    for score in scores:
        if score < min_val:
            min_val = score
        total += score

    total -= min_val
    return total / (len(scores) - 1)
```

- ARRAY con 1 solo elemento
- T1({1}; 1) loops 1
- T2({1,2,3}; 2) loops >1
- T3({}; ?) loops 0

SoftEng

mi do un
valore enorme:
total/0.



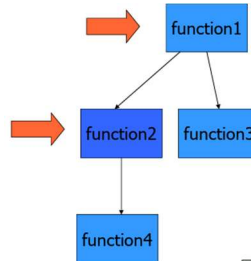
E' comodo trasformare il white box testing in un CONTROL FLOW GRAPH (grafo che rappresenta tutti i possibili percorsi di esecuzione del programma):

- Nodi → decisioni, istruzioni atomiche
- Edge → passaggio da una istruzione alla successiva
- Block → insieme di istruzioni che vengono eseguite insieme

Integration test

Non basta verificare che ogni singola funzione sia corretta, bisogna anche verificare che **funzioni corrette riescano a collaborare correttamente tra loro**.

GRAFO DELLE DIPENDENZE: (Dependency graph), ossia il grafo delle dipendenze tra le funzioni per capire in quale ordine conviene eseguire i test di integrazione. Le funzioni si chiamano a vicenda. Prima individuiamo le funzioni che non hanno dipendenze e le



testiamo singolarmente (unit testing).

in questo caso function 3 e function 4. Quindi prima si testano le unità singolarmente (unit testing) e poi si cominciano ad unire poco alla volta, non bisogna mai prendere tutto il programma e provarlo in una volta sola. Prima verifico che ogni funzione, da sola, sia corretta. Poi collego due funzioni e verifico che comunichino correttamente. Successivamente aggiungo una terza funzione. Poi una quarta. Ogni volta eseguo nuovamente i test. Così, se compare un errore, so che è stato introdotto proprio nell'ultima integrazione che ho effettuato. Evitiamo big bang integration: testiamo singolarmente tutte le funzioni e poi solo alla fine le colleghiamo insieme e lanciamo il programma... non funziona...il problema potrebbe essere ovunque.

Ci sono 2 strategie principali:

- **BOTTOM-UP** → si parte dalle funzioni più in basso, quelle che non dipendono da nessun'altra. Nell'esempio sopra parto da Function 3 e function 4. La prima cosa che faccio è testarle da sole con unit testing. Quando sono sicuro che funzionano allora collego function 2 con function 4. A questo punto non sto più verificando soltanto Function2, ma sto verificando anche la comunicazione tra Function2 e Function4. Se durante questo test compare un errore, posso ragionare in questo modo: "Function4 l'ho già testata prima ed era corretta. Quindi il problema non può essere lì. Rimangono solo due possibilità: o c'è un errore dentro Function2 oppure c'è un errore nell'interazione tra Function2 e Function4." Infine collego anche Function1. Quando eseguo l'ultimo test, so già che tutto ciò che sta sotto funziona. Se compare un errore, sarà molto probabilmente dentro Function1 oppure nel modo in cui comunica con Function2 e Function3. Questo rende il debugging molto più semplice.
- **TOP-DOWN** → si parte dalla funzione più in alto, cioè quella principale. Nel nostro esempio si parte da Function1. Il problema, però, è che Function2 e Function3 magari non sono ancora state sviluppate. Creo delle funzioni finte che simulano il comportamento di quelle reali. Queste funzioni si chiamano **Stub**. Uno Stub è

semplicemente una funzione temporanea che restituisce un risultato plausibile. Per esempio, se Function1 chiama una funzione che dovrebbe leggere il database, lo Stub potrebbe limitarsi a restituire sempre un utente fittizio. Così posso già verificare se la logica di Function1 è corretta, anche se il resto del sistema non esiste ancora. Quando Function2 viene completata, elimino il suo Stub e la sostituisco con la funzione vera.

Il DRIVER serve nel BOTTOM UP → è un piccolo programma temporaneo che simula il chiamante della funzione. "finge" di essere la funzione superiore che ancora non esiste.

Lo **Stub**, invece, serve nel Top-Down. È una funzione temporanea che simula il comportamento di una funzione inferiore che ancora non è stata sviluppata.

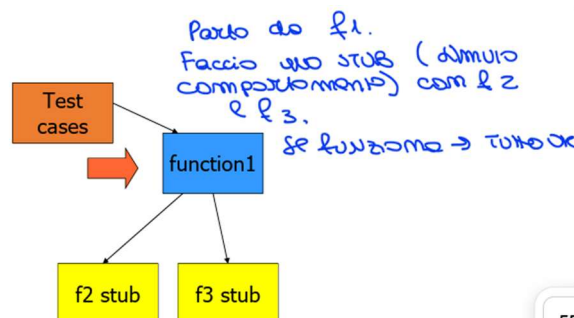
VANTAGGI TOP DOWN: trova problematiche architetturali. Si può già vedere se i moduli principali collaborano nel modo corretto e si ottiene rapidamente una versione funzionante, anche se incompleta. Lo svantaggio è che bisogna scrivere molti Stub.

VANTAGGI BOTTOM – UP: permette di iniziare subito dai componenti di basso livello e di verificarli molto accuratamente. Lo svantaggio è che, non esistendo ancora i moduli superiori, bisogna costruire dei Driver per poterli eseguire.

Nella pratica si adotta un mix tra bottom up e top down.

Top down

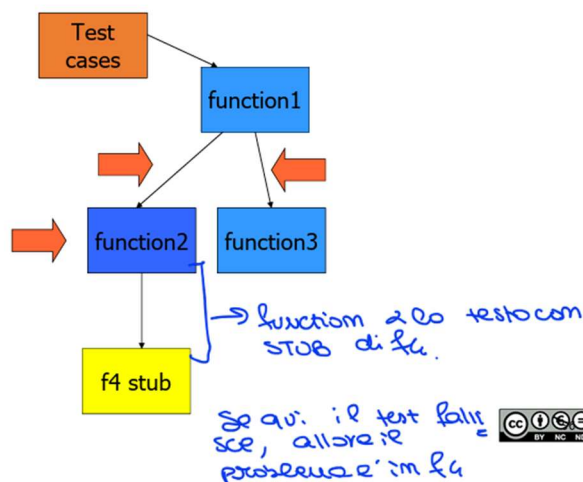
- Step 1



55

Top down

- Step 2



- il **Driver** simula **chi chiama**;

- lo **Stub** simula **chi viene chiamato**.

Se invece abbiamo delle DIPENDENZE CIRCOLARI (classe A utilizza classe B e contemporaneamente la classe B utilizza la classe A, si crea un ciclo). Diventa impossibile stabilire da quale delle due partire per l'integrazione, perché ognuna ha bisogno dell'altra. Le possibili soluzioni sono due:

- considerare tutto il ciclo come un unico blocco e testarlo insieme;
- oppure, soluzione preferibile, modificare il progetto del software eliminando la dipendenza circolare.

Fixture

FIXTURE = preparazione dell'ambiente di test, tutto ciò che viene preparato prima che inizi il test. Quando eseguiamo un test, il codice da testare difficilmente può partire da solo perché magari ha bisogno di un utente già registrato /db già popolato / file già creato... oer questo ogni volta preparare tutto dentro ogni test sarebbe duplicazione di codice, infatti vengono introdotte le fixture. Il test deve partire da uno stato conosciuto, stabile e riproducibile. La fixture può contenere dati, configurazioni, condizioni dell'ambiente....

la fixture viene eseguita prima del test vero e proprio. Ogni test deve iniziare **sempre dalla stessa situazione iniziale**, perché altrimenti due esecuzioni dello stesso test potrebbero produrre risultati diversi. L'obiettivo è **non mischiare il codice di preparazione con il codice del test**. Il test dovrebbe essere il più leggibile possibile. Le fixture sono importanti perché ci evitano duplicazione di codice, facilitano la manutenzione (qualora dovessimo apportare delle modifiche, basterebbe modificare una sola funzione di preparazione), e perché permettono ai test di concentrarsi **solo sulla logica che vogliamo verificare**. Quando leggo un test voglio capire immediatamente cosa sta controllando. Codice più pulito, leggibile e riutilizzabile. Solitamente vengono usate per preparare gli input data per un test, inizializzare risorse condivise da + tests, configurare l'ambiente di esecuzione...

```
@pytest.fixture(scope="function", autouse=False)
def user_id():
    return 42

def test_user_id(user_id):
    assert user_id == 42
```

viene passata la fixture come parametro della funzione di test. Pytest risolve automaticamente la fixture e inject il valore all'interno del test.

Lo scope può essere: function (di default)/ class/module/package, session.

Autouse = True → la fixture viene eseguita automaticamente per tutti i test che la vedono.
(autouse è False di default).

A volte non basta preparare qualcosa (preparare l'ambiente di test → fase di setup), ma dobbiamo anche rimuoverla quando il test è terminato per riportare il sistema in uno stato pulito (tipo la chiusura di un file) → fase di teardown. Questo permette al test successivo di partire da un ambiente pulito. Se la fixture serve solo per il setup allora nel codice utilizziamo return:

```
@pytest.fixture
```

```
def numero():
```

```
    return 10
```

Se invece deve anche ripulire una risorsa, usiamo anche yield, che divide il codice in due parti: la parte prima di yield per preparare il test e la parte dopo yield per rimuovere la risorsa. La parte di teardown parte quando il test è completamente terminato.

```
@pytest.fixture
def opened_file():
    file = open("test.txt", "w")
    yield file
    file.close()

def test_file_is_open(opened_file):
    assert opened_file.closed is False
```

Mocking (oggetto finto che prende il posto di un oggetto reale durante un test)

il mocking è una tecnica di testing che consiste nel sostituire le dipendenze reali del codice che stiamo testando con oggetti o funzioni sostitutive, completamente controllate dal test.

Con dipendenza intendiamo questo:

```
def salva_utente():
```

database.save() → qui c'è una dipendenza dal database. Però noi vogliamo solo testare la funzione e non ci interessa verificare se il database funziona o meno. Il mocking infatti ci permette di **eseguire esclusivamente la logica che vogliamo verificare**, senza coinvolgere tutto il resto del sistema. In pratica isoliamo il componente da testare. Il mock controlla soprattutto le interazioni tra le funzioni.

Il mock permette di sostituire una reale dipendenza, di definire i valori di ritorno di una funzione, di simulare eccezioni o condizioni anomale, vuole isolare la logica di test da tutto il resto. Si usa quando vogliamo ad esempio forzare diversi valori di ritorno per testare il comportamento di alcuni branches, o se vogliamo testare una funzione che chiama un'altra funzione. Possono essere mockate standalone function, metodi o intere istanze di un oggetto.

LIBRERIA: unittest.mock. Ci fornisce la classe Mock per creare dei test doubles controllati.

```
class UserRepository: 5 usages
    def find_by_id(self, user_id): 8 usages (8 dynamic)
        pass

@pytest.fixture
def user_repository_mock():
    return Mock(spec=UserRepository)

def test_user_repository(user_repository_mock):
    assert hasattr(user_repository_mock, "find_by_id")

    with pytest.raises(AttributeError):
        user_repository_mock.find_by_name
```

Gli stiamo passando il prototipo della classe. Con spec indichiamo che il Mock deve comportarsi come se fosse un UserRepository, anche se non lo diventa veramente, ma ne copia l'interfaccia e quindi i metodi disponibili (in questo caso posso chiamare solo find by id), questa cosa permette di individuare subito gli errori (AttributeError).

Se vogliamo che restituisca un valore fisso, usiamo uno stub: indipendentemente da cosa passiamo a find_by_id() avremo sempre "id": 1 e "name": "John".

```
@pytest.fixture
def user_repository_stub():
    repository = Mock(spec=UserRepository)
    repository.find_by_id.return_value = {"id": 1, "name": "John"}
    return repository

def test_get_user_name_output(user_repository_stub):
    user = user_repository_stub.find_by_id(1)
    assert user["name"] == "John"
```

```
@pytest.fixture
def user_repository_stub():
    repository = Mock(spec=UserRepository)
    repository.find_by_id.side_effect = [
        LookupError("Temporary error"),
        {"id": 1, "name": "John"},
    ]
    return repository

def test_side_effect_with_multiple_results(user_repository_stub):
    with pytest.raises(LookupError):
        user_repository_stub.find_by_id(1)

    assert (user_repository_stub.find_by_id(1) ==
            {"id": 1, "name": "John"})
```

Se vogliamo simulare il lancio di eccezioni o restituire valori diversi, usiamo side_effect. Il Mock può essere usato anche come Spy, se vogliamo controllare come è stato utilizzato e non solo cosa restituisce il Mock, in questo caso registrerà tutte le chiamate ricevute.

```

@pytest.fixture
def user_repository_mock():
    return Mock(spec=UserRepository)

def test_get_user_name_interactions(user_repository_mock):
    user_repository_mock.find_by_id(1)
    user_repository_mock.find_by_id.assert_called_with(1)
    user_repository_mock.find_by_id.assert_called_once_with(1)

```

così verifichiamo anche le interazioni e non solo il risultato finale.

Patch() → usato per rimpiazzare automaticamente un oggetto reale con un Mock, solo temporaneamente.

patcher = patch("UserRepository", new=mocked_class) → quando qualcuno usa UserRepository, invece usa mocked_class.

patcher.start() → inizia la sostituzione. patcher.stop() ripristina l'oggetto originale.

```

@pytest.fixture(scope="module", autouse=True)
def patch_user_repository():
    mocked_class = Mock()
    mocked_class.return_value.find_by_id.return_value = {
        "id": 1,
        "name": "John",
    }
    patcher = patch("UserRepository", new=mocked_class)
    patcher.start()
    yield
    patcher.stop()

def test_repository_is_patched_in_this_module():
    repository = UserRepository()
    assert repository.find_by_id(1) == {"id": 1, "name": "John"}

```

FARE IL SETUP
→ rimpiazzo le istanze di UserRepository con le istanze di Mock.

TESTING MULTILAYER APPLICATION

Durante i test non bisogna mai utilizzare il database dell'applicazione reale, perché i test potrebbero modificare o cancellare dati persistenti. Per questo motivo i test devono essere eseguiti su un database isolato e dedicato esclusivamente al testing. Esistono due strategie principali: usare dei mock della persistenza, tipico dei test unitari, oppure utilizzare un database di test (ad esempio SQLite in-memory), che riproduce il comportamento reale ed è usato soprattutto nei test di integrazione ed end-to-end.

🔍 **Mock della Persistence Layer:** si sostituiscono i metodi del repository (come find(), save(), delete()) con oggetti simulati. In questo modo si testa soltanto la logica dell'applicazione senza configurare un database reale. È la soluzione tipica dei **test unitari**, perché è veloce e permette di isolare il componente da testare.

🔍 **Test Data Source:** si utilizza un database reale dedicato ai test, spesso SQLite in-memory. Il database viene creato da zero per ogni esecuzione del test e viene eliminato al termine. Questa soluzione è più lenta ma molto più realistica, quindi viene utilizzata nei **test di integrazione** e **end-to-end**.

TEST CASE ISOLATION: Ogni test deve essere completamente indipendente dagli altri. Un test non deve dipendere dai dati creati da test precedenti e non deve lasciare modifiche permanenti che possano influenzare l'esecuzione dei test successivi. In questo modo ogni test può essere eseguito in qualsiasi ordine ottenendo sempre lo stesso risultato.

READ ONLY TEST: I test che eseguono soltanto operazioni di lettura possono condividere un database iniziale già popolato (fixture database). Questo è possibile solo se i test non modificano i dati, non dipendono dall'ordine di esecuzione e il contenuto iniziale del database rimane sempre identico. Questo approccio riduce il tempo necessario per preparare l'ambiente di test. Quando un test inserisce, modifica o elimina dati, è necessario ripristinare il database al termine dell'esecuzione. Questo può essere fatto effettuando il **rollback** della transazione, ricreando lo schema del database oppure ricreando completamente un database in-memory. Lo scopo è garantire che ogni test parta sempre dalle stesse condizioni iniziali. Le **Entity** rappresentano semplicemente le tabelle del database e contengono solo dati, relazioni e metadati. Sono dei semplici data holder e per questo non devono essere testati. La logica sta nei SERVIZI e REPOSITORY.

L'applicazione è organizzata in livelli:

Interface (Routes) → Controller → Service → Repository → Model (Entity) → Database

Ogni livello ha una responsabilità specifica:

- **Interface (Routes):** riceve le richieste HTTP e le inoltra al Controller. Non adatte a test unitari ma più a test di integrazione per verificare autenticazione, che i codici di risposta HTTP funzionino.
- **Controller:** coordina il flusso dell'applicazione chiamando i Service. Nei **test unitari** si sostituiscono i Service con dei mock e si verifica che vengano chiamati correttamente, che i parametri siano corretti e che risultati ed errori vengano propagati nel modo previsto. Si possono eseguire anche test di integrazione per verificare il comportamento del Controller con componenti reali.
- **Service:** contiene la business logic. Nei test unitari si simulano Repository e altri Service tramite mock, verificando che le regole di business siano applicate correttamente. Per i flussi più importanti vengono eseguiti anche test di integrazione.
- **Repository:** gestisce l'accesso al database tramite ORM. Se implementa logica personalizzata, va testato con unit e integration.
- **Model/Entity:** rappresenta i dati. No testing. Contiene DTO (Data transfer objects , non vanno testati).
- **Database:** memorizza i dati persistenti.

END TO END TESTING

Il System Testing consiste nel testare l'intero sistema una volta completata l'integrazione di tutti i componenti. L'obiettivo è verificare che il software soddisfi i requisiti, sia funzionali sia non funzionali. A differenza dei test unitari e di integrazione, qui non si testa più il singolo componente, ma il sistema nel suo complesso. Si basa sul documento dei requisiti e si focalizza sui requisiti funzionali e non funzionali.

La copertura del System Testing prevede:

- **almeno un test per ogni requisito funzionale;**
- **almeno un test per ogni scenario d'uso;**
- **almeno un test per ogni requisito non funzionale.**

ACCEPTANCE TESTING: TEST eseguito dal cliente per verificare che il sw soddisfi quanto stabilito nel contratto (casi di test generalmente definiti dal cliente). **CHANGE REQUEST:** variazioni che implementa al di fuori del contratto, richiesta di modifica ai requisiti o al sistema, può portare ad una nuova versione del sistema.

BETA TESTING: A differenza dell'Acceptance Test, il Beta Test non serve ad accettare formalmente il prodotto, ma a raccogliere feedback dagli utenti. Il Beta Testing viene utilizzato soprattutto nei prodotti destinati al mercato di massa. Una versione quasi definitiva del software viene distribuita a un gruppo limitato di utenti reali che la utilizza in condizioni normali. L'obiettivo è individuare problemi che non erano emersi durante lo sviluppo prima della distribuzione ufficiale. Test cases informali.

Il System Testing può essere eseguito in tre modi principali:

- attraverso la **GUI** (interfaccia grafica);
- attraverso **scenari End-to-End**, simulando un flusso completo dell'utente;
- attraverso le **API**, interagendo direttamente con il backend.

API TESTING:

L'**API Testing** consiste nel testare direttamente i servizi del backend, senza utilizzare la GUI (Graphical User Interface). Si inviano direttamente delle richieste HTTP al server e si controlla la risposta. Ad esempio, invece di aprire un'app bancaria e premere il pulsante "Visualizza saldo", il test invia direttamente una richiesta HTTP al servizio che restituisce il saldo. L'obiettivo è verificare il comportamento del sistema **a livello di servizio (service level)**. Rispetto a GUI TESTING è più veloce, controllabile (no bottoni sbagliati cliccati) e più preciso (permette di controllare i messaggi di errore, dati restituiti in formato json, codici di stato HTTP).

API (Application Programming Interface): È un'interfaccia che permette a due sistemi software di comunicare. ES: App Android → richiesta HTTP → REST API → DATABASE (L'app non legge direttamente il db, ma invia una richiesta alla REST API che recupera i dati dal db e restituisce la risposta).

Come funziona una REST API? Client invia una richiesta (GET /users/15), il server controlla la richiesta, interroga il db ed esegue la logica, restituisce una risposta con uno status code e i dati in JSON. REST è stateless perché ogni richiesta è indipendente dalle precedenti (il server non ricorda automaticamente ciò che è successo prima). Le risorse sono identificate tramite URI (indirizzo univoco: /users) (ogni info nel sistema è una risorsa). Ogni richiesta usa un metodo HTTP: GET per recuperare i dati, POST per creare nuovi dati, PUT per aggiornare dati esistenti, DELETE per eliminare dati. Si utilizza POSTMAN (ogni richiesta rappresenta un test case e quindi per creare una richiesta serve URL, metodo HTTP, eventuali parametri, eventuale body). Un insieme di richieste raggruppate si chiama COLLECTION, eseguite tramite COLLECTION RUNNER, in modo automatico, come una test suite completa.

GUI TESTING:

Il **GUI Testing (Graphical User Interface Testing)** consiste nel verificare il corretto funzionamento di un'applicazione **interagendo con la sua interfaccia grafica**, proprio come farebbe un utente reale. Si tratta di un **black-box test**: il tester non conosce il codice sorgente, ma verifica il comportamento del sistema utilizzando pulsanti, menu, finestre, campi di testo e tutti gli altri elementi dell'interfaccia. L'obiettivo è controllare che l'utente riesca a utilizzare correttamente il software e che le funzionalità siano accessibili e funzionino come previsto.

Viene usato per 3 scopi:

- **FUNCTIONAL TESTING**: Serve a verificare che le funzionalità dell'applicazione siano corrette dal punto di vista dell'utente
- **2. Look & Feel Verification**: IL sw deve apparire come previsto (colori, layout, posizione elementi)
- **COMPATIBILITY TESTING**: si verifica che l'applicazione funzioni correttamente su dispositivi e ambienti diversi. Tutti gli utenti devono avere la stessa esperienza d'uso.

Si usa: approccio MANUALE (junior tester che fa manualmente tutti gli scenari di test). E' semplice ma è error prone e richiede molto tempo. Oppure SCRIPTED (si usano degli script automatici che simulano le azioni dell'utente). Veloce, ma Quando l'interfaccia cambia, anche gli script devono essere aggiornati. Per questo motivo possono diventare costosi da mantenere.

PROBLEMI PRINCIPALI GUI TESTING:

- **FLAKINESS**: Un test è **flaky** quando, sullo stesso software e senza modifiche, a volte passa e a volte fallisce. Il problema è che diventa difficile capire se il software contiene davvero un difetto oppure se il fallimento è casuale. Le cause possono essere: tempi di esecuzione variabili, rete lenta...
- **FRAGMENTATION**: La **fragmentation** si verifica quando lo stesso test produce risultati diversi su ambienti differenti. Può dipendere da browser diversi (BROWSER FRAGMENTATION), versioni diverse del SO (SW FRAGMENTATION), dispositivi diversi (telefono, tablet, HARDWARE FRAGMENTATION)
- **FRAGILITY**: Un test è **fragile** quando smette di funzionare dopo una nuova versione dell'applicazione, anche se la funzionalità è ancora Corretta. Ad esempio, se il pulsante "Login" viene semplicemente spostato o rinominato, il test automatico può fallire pur senza alcun errore nel software.

- Il problema quindi non è l'applicazione, ma il fatto che il test è troppo dipendente dall'interfaccia grafica

VARIE TECNICHE:

- CAPTURE & REPLAY: il tester esegue una volta le azioni manualmente e il tool le registra. Poi il tool **ripete automaticamente** le stesse azioni. Semplice ma fragile (se sposto il pulsante Login il test non funziona +).
- MODEL-BASED GUI TESTING: prima costruisco un **modello** della GUI (ad esempio un grafo delle pagine o una macchina a stati). Da questo modello il tool genera automaticamente molti test. Alta copertura ma costoso e complicato
- MONKEY TESTING: il programma clicca casualmente pulsanti, menu e campi di testo. (creazione di input sequences per navigare l'interfaccia). Trova bug inattesi ma non verifica una funzionalità precisa.

Per fare un test automatico bisogna dire al programma **quale elemento cliccare**. Quindi o si clicca in una posizione dello schermo, o si cerca l'elemento tramite una proprietà (ID HTML) → PROPERTY BASED (+ usato) (USO DEI LOCATOR: ganci per identificare gli elementi all'interno della pag) o si riconosce direttamente l'immagine del pulsante.

Si può misurare COVERAGE in vari modi.

- **Requirements Coverage:** quanti requisiti ho verificato.
- **Functional Coverage:** quante funzionalità ho testato.
- **Page Coverage:** quante pagine dell'app ho visitato.
- **Widget Coverage:** quanti elementi grafici (bottoni, menu, textbox...) ho usato.
- **Path Coverage:** quanti percorsi dell'utente ho provato.

Noi usiamo Selenium .

MUTATION TESTING:

Il **Mutation Testing** è una tecnica che serve a valutare la qualità dei test, non del programma. L'idea è molto semplice: invece di cercare bug nel codice, si introducono volontariamente dei piccoli errori (chiamati **mutanti**) e si verifica se la test suite è in grado di individuarli. Se i test falliscono, significa che sono abbastanza efficaci; se invece continuano a passare, vuol dire che non stanno verificando correttamente quel comportamento.

MUTANTE: variante del mio programma originale per simulare un possibile errore di programmazione.

Si sceglie il codice da analizzare, si generano automaticamente i mutanti, si esegue la stessa test suite su ogni mutante e infine si osserva il risultato. Se almeno un test fallisce, il mutante viene detto **killed** (ucciso), perché i test hanno rilevato l'errore introdotto. Se tutti i test passano, il mutante è **survived** (sopravvissuto), perché i test non si sono accorti della modifica. In alcuni casi può verificarsi anche un **timeout**, cioè il mutante provoca un'esecuzione troppo lunga (ad esempio un ciclo infinito). Il vantaggio principale del mutation testing è che permette di capire **dove i test sono deboli**. Serve a verificare se **i test sono davvero in grado di trovare bug**.

I **test smells** sono cattive pratiche nella scrittura dei test che li rendono difficili da capire, mantenere o affidabili. Non fanno necessariamente fallire i test, ma ne riducono la qualità.

Resource Optimism → Il test **assume che una risorsa esterna sia sempre disponibile** (file, database, rete), senza verificarlo.

Assertion Roulette → Il test contiene **molte assert senza messaggi esplicativi**: quando fallisce, è difficile capire quale verifica è fallita.

Magic Number Test → Il test usa **numeri o valori "magici"** (es. 42, 999) senza spiegarne il significato, rendendo il test poco leggibile e difficile da mantenere.

❓ **Test Run War**: i test **interferiscono tra loro** e il risultato cambia in base all'ordine di esecuzione, perché condividono stato o dati.

❓ **Lazy Test**: **più test verificano la stessa cosa** usando le stesse fixture, senza aumentare la copertura; sono test ridondanti.

❓ **Eager Test**: **un solo test verifica troppi comportamenti** contemporaneamente; se fallisce è difficile capire quale parte è sbagliata.

ARCHITETTURA:

L'architettura software definisce come il sistema è organizzato in componenti e come questi comunicano tra loro. Un buon progetto architetturale rende il sistema più manutenibile perché limita l'impatto delle modifiche: se ogni componente ha responsabilità ben definite e poche dipendenze dagli altri (alta coesione e basso accoppiamento), è possibile modificare una parte del sistema senza influenzare tutto il resto. Questo è fondamentale perché, secondo **le leggi di Lehmann, il software evolve continuamente e la sua complessità tende ad aumentare nel tempo.**

La **modularità** consiste nel suddividere il sistema in moduli, componenti o servizi indipendenti, ognuno con una responsabilità specifica. Ogni modulo dovrebbe avere **alta coesione** (le sue funzionalità sono strettamente correlate) e **basso accoppiamento** (dipende il meno possibile dagli altri moduli). Questa suddivisione rende il software più semplice da sviluppare, testare, riutilizzare e mantenere.

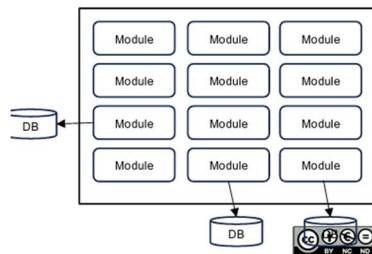
L'**architettura software** è la struttura data al sistema da chi lo progetta. Definisce come il sistema è suddiviso in componenti, come questi sono organizzati e come comunicano tra loro, per facilitare sviluppo, deployment, funzionamento e manutenzione.

L'obiettivo principale dell'architettura software è **mantenere il sistema utile e di valore per gli utenti nel tempo, minimizzando il costo complessivo del suo ciclo di vita e massimizzando la produttività degli sviluppatori**. Per raggiungere questi obiettivi, l'architettura deve rendere il sistema **facile da comprendere, sviluppare, distribuire, modificare e mantenere**. In questo modo il software può evolvere nel tempo con minori costi di manutenzione e maggiore facilità di adattamento ai cambiamenti.

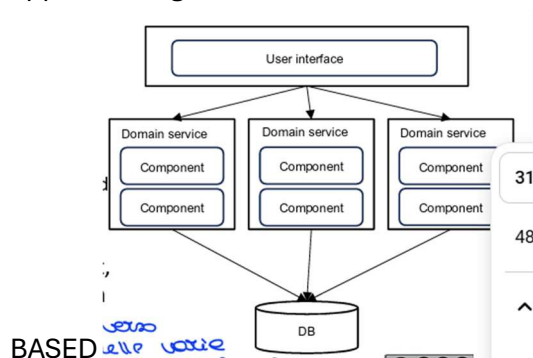
Un'architettura **monolitica** è un'architettura in cui **tutto il codice dell'applicazione viene distribuito come un'unica unità di deployment** (un solo eseguibile o pacchetto). Tutti i componenti fanno parte dello stesso processo e comunicano tramite chiamate interne, senza

utilizzare la rete. È semplice da sviluppare e gestire, ma per modificare o scalare una parte è necessario ridistribuire l'intera applicazione. Può essere:

- **LAYERED ARCHITECTURE (n-tier)** : sw suddiviso in livelli indipendenti. ognuno con una responsabilità precisa. (Presentation Layer → gestisce l'interfaccia utente. Business Layer → contiene la logica applicativa. Persistence Layer → gestisce l'accesso ai dati. Database Layer → memorizza i dati.). Adatta per app piccole, semplici, con responsabilità separate, poco tempo o budget. No per sistemi che cambiano continuamente. Ha una struttura chiara e facile da sviluppare ma è poco flessibile e non scala
- **MODULAR MONOLITH**: viene distribuito come **un'unica applicazione**, ma internamente è diviso in **moduli indipendenti**, ognuno dedicato a una specifica funzionalità di business. Es: modulo ordini, modulo clienti. Tutte le funzionalità simili messe dentro stesso modulo. Possono comunicare tramite mediator o con db condiviso. **SVANTAGGIO**: con il tempo i moduli possono diventare troppo dipendenti tra loro (tight coupling), no scalabile, codice può diventare molto grande. Si usa all'inizio dei progetti o con team piccoli. No network overhead e semplice.



- In un'**architettura distribuita**, invece, il sistema è composto da più unità di deployment indipendenti che comunicano tramite la rete (ad esempio con API REST o messaggi). Questo permette di distribuire e scalare ogni servizio separatamente, ma introduce maggiore complessità operativa, comunicazione di rete e necessità di monitoraggio. Ogni servizio svolge una specifica funzione e può essere sviluppato, distribuito, aggiornato e scalato indipendentemente dagli altri. Questo rende il sistema più flessibile e adatto ad applicazioni grandi e in continua evoluzione. Es: event driven, **MICROSERVIZI**, **SERVICE-**

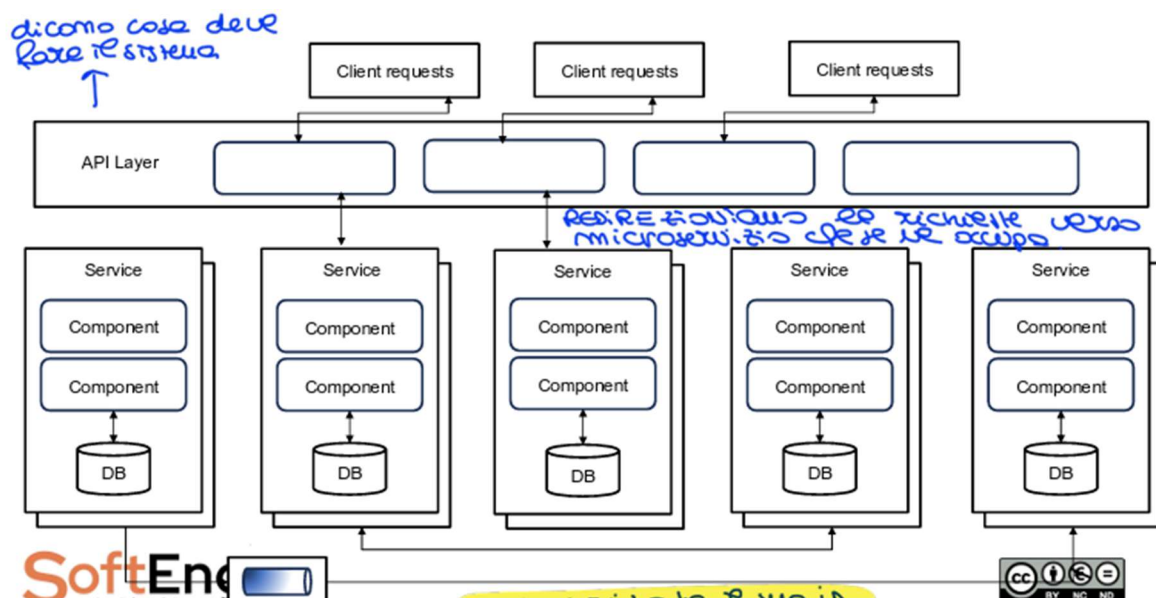


L'**architettura a microservizi** è un tipo di architettura distribuita in cui l'applicazione non è più un unico programma, ma viene suddivisa in tanti piccoli servizi indipendenti. Ogni microservizio realizza una specifica funzionalità di business, ad esempio gestione utenti, ordini, pagamenti o catalogo, ed è autonomo: possiede il proprio codice, spesso il proprio database e può essere

sviluppato, aggiornato e distribuito senza modificare gli altri servizi. I microservizi comunicano tra loro attraverso la rete usando protocolli standard, come REST e HTTP, esponendo delle API.

L'idea deriva dal **Domain-Driven Design (DDD)**: ogni servizio rappresenta un preciso **bounded context**, cioè un'area ben delimitata del dominio applicativo. Il client non comunica direttamente con tutti i servizi. Generalmente invia la richiesta ad un **API Layer** (o API Gateway), che riceve la richiesta e la inoltra al microservizio che deve occuparsene.

Ogni microservizio contiene: la propria logica; i propri componenti; molto spesso anche il proprio database. In questo modo i servizi sono completamente separati e possono evolvere indipendentemente.



Un microservizio deve essere soprattutto **piccolo** e **autonomo**. Si separano secondo le funzionalità di business (business boundaries), così che ogni servizio si occupa di una sola area del dominio applicativo riducendo le dipendenze. Così ogni servizio può evolvere senza influenzare gli altri. Sam Newman dice che un microservizio dovrebbe essere:

- abbastanza piccolo da poter essere gestito da un piccolo team (circa 5-10 persone);
- abbastanza piccolo da poter essere riscritto in poche settimane se necessario.

L'idea è che ogni servizio faccia una sola cosa e la faccia bene. Ha una responsabilità ben precisa. Autonomo invece significa: Ogni microservizio viene eseguito come processo indipendente. Comunica con gli altri solo tramite chiamate di rete (REST, HTTP, gRPC, ecc.) e mette a disposizione delle **API**. Questo permette il **decoupling** (disaccoppiamento): posso modificare o distribuire un servizio senza obbligare gli altri servizi a cambiare, purché le API rimangano compatibili.

VANTAGGI: scalabilità (scalo solo il servizio necessario quando serve, con monolite scalo tutto insieme, anche le parti che non dovrebbero), deploy indipendenti (in un monolite anche una piccola modifica richiede il build, test e deploy dell'intera applicazione, nei microservizi invece solo il servizio interessato), organizzazione dei team (ogni microservizio può essere sviluppato da

un piccolo team indipendente e autonomo), riusabilità del codice, sostituibilità (se un microservizio diventa obsoleto, posso sostituire solo quello, senza riscrivere tutta l'app).

SVANTAGGI: Dato che comunicano attraverso la rete, questo introduce overhead (latenza). Nel monolite invece i componenti comunicano tramite chiamate di funzioni che sono + veloci. Introducono maggiore complessità .

CLEAN ARCHITECTURE: La **Dependency Rule** afferma che **le dipendenze del codice devono puntare verso l'interno**, cioè verso i livelli che contengono le regole di business (policy) e non verso quelli che contengono i dettagli tecnici (database, framework, interfaccia grafica). In questo modo la logica di business rimane indipendente dalle tecnologie utilizzate.

La **Layered Architecture** è meno adatta per sistemi che richiedono **alte prestazioni o che sono molto dinamici**. Questo perché ogni richiesta deve attraversare tutti i livelli dell'architettura, aumentando l'overhead e rendendo più difficile ottenere elevate prestazioni o modifiche frequenti.

Docker

Docker è una piattaforma software che semplifica il processo di **building, running, managing e distributing** delle applicazioni containerizzate. Virtualizza il sistema operativo della macchina su cui è installato, permettendo un deployment efficiente delle applicazioni. Il problema affrontato da Docker è il seguente:

- uno stesso server può ospitare più applicazioni;
- ogni applicazione può utilizzare framework differenti (ad esempio Node.js, Spring o Flask);
- le applicazioni possono richiedere versioni diverse dello stesso framework;
- ogni applicazione può avere dipendenze differenti.

Senza Docker sarebbe difficile installare contemporaneamente più versioni di Node.js, Java o Python sulla stessa macchina, poiché potrebbero entrare in conflitto. Docker risolve questo problema creando **container**, cioè entità logiche isolate tra loro. Senza Docker dovremmo avere + macchine fisiche separate o avere un'unica macchina fisica con + virtual machines.

VANTAGGI: Così quindi riduciamo i costi hw, abbiamo un minore utilizzo di memoria e disco, eliminiamo la necessità di utilizzare molte macchine fisiche o virtual machines, possiamo eseguire applicazioni con framework diversi sullo stesso host e c'è un avvio + rapido.

SVANTAGGI:

Ogni container contiene:

- l'applicazione;

- il framework necessario;
- tutte le librerie;
- tutte le dipendenze richieste.

In questo modo più applicazioni con requisiti completamente differenti possono essere eseguite sullo stesso host senza interferire tra loro. I container condividono solamente il **kernel del sistema operativo host**, mentre mantengono separati processi, rete e file system.

Docker e le Virtual Machine consentono entrambe di isolare le applicazioni, ma lo fanno in maniera diversa. Ogni VM contiene: applicazione, librerie e sistema operativo guest completo. Tutte le VM sono gestite da un Hypervisor. Ogni VM possiede un proprio SO, richiede molte risorse, occupa molto spazio e impiega tanto tempo ad avviarsi. Invece con docker non esiste un SO all'interno del container, tutti i container condividono il kernel del SO host, e sono gestiti da un DOCKER ENGINE. I container quindi sono isolati ma condividono SO host, sono molto + leggeri, veloci da avviare e – costosi in termini hw.

I container Docker sono ambienti isolati ma spesso devono comunicare tra di loro o con l'esterno, quindi usano:

- bridge network: rete predefinita di Docker. Rete virtuale interna all'host di Docker. I container che appartengono alla stessa bridge, possono comunicare tra loro usando il nome del container come indirizzo, grazie al servizio DNS interno a Docker.
- Host network: il container utilizza direttamente gli indirizzi IP e le porte della macchina fisica (riduzione overhead), non usa una rete virtuale separata, ma condivide direttamente lo stack di rete dell'host. Però c'è minore separazione e maggiore rischio di conflitti.
- Overlay net: permette a container collocate su host docker diversi di comunicare.

SVANTAGGI DOCKER:

- Peggioramento delle performance dovuto alla condivisione delle risorse dell'host tra container.
- Docker richiede un training iniziale
- Docker non può ospitare applicazioni con diversi SO sulla stessa macchina perché i container condividono il kernel dell'host (container Linux e container Windows non possono normalmente essere eseguiti sullo stesso Docker Host)

DOCKER ENGINE: Il Docker Engine è il componente centrale della piattaforma Docker. È il software installato sull'host che gestisce le operazioni complessive del sistema Docker. Può: creare e gestire le immagini, creare, avviare o arrestare i container, gestire le reti e i volumi, comunicazione con utente e app esterne.

Comprende: Server, REST API, Client (interfaccia a riga di comando usata dall'utente, quando viene eseguito un comando docker ps → client invia una richiesta a daemon in background del server tramite la REST API, questo recupera l'elenco dei container e restituisce il risultato al client ad es).

DOCKER IMAGE: template statico contenente: il codice dell'applicazione; il runtime; le librerie; le dipendenze; la configurazione necessaria; i comandi necessari per avviare l'applicazione. L'immagine può essere considerata il progetto o la fotografia immutabile dell'applicazione e del suo ambiente. Non è un processo in esecuzione. Serve come base per creare uno o più container.

DOCKER CONTAINER: un'istanza in esecuzione di una Docker Image. L'immagine è quindi statica, mentre il container è dinamico. Quando si avvia un'immagine, Docker crea un container nel quale viene eseguita l'applicazione. Dalla stessa immagine possono essere creati più container indipendenti.

Docker HUB → repository online utilizzato per memorizzare e distribuire Docker Image (rendere disponibile la mia ad altri e vedere quella degli altri).

Il **GitHub Container Registry**, indicato come GHCR, svolge una funzione simile, ma è integrato nell'ecosistema GitHub. Permette di: ospitare e gestire immagini Docker;

Una docker image è costituita da una sequenza di layer sovrapposti. Durante la build, ogni istruzione significativa del Dockerfile genera un layer, si tratta di READ-ONLY LAYER (non possono essere modificati dopo la creazione). Alla fine della build vengono sovrapposti e presentati come un unico file system. Questo meccanismo viene indicato nelle slide come **Union File System**. I layer delle immagini sono immutabili. Se viene modificata un'istruzione del Dockerfile, Docker non cambia il vecchio layer, ma ne crea uno nuovo. I layer precedenti che non sono cambiati possono essere riutilizzati grazie alla cache. Questo riduce il tempo richiesto per costruire nuovamente l'immagine. Quando viene creato un container a partire da un'immagine, Docker mantiene tutti i layer Read-Only dell'immagine e aggiunge sopra di essi un nuovo layer **Read-Write**. Questo nuovo livello è chiamato **container layer**. Tutte le modifiche eseguite durante il funzionamento del container vengono salvate in questo layer, per esempio: creazione di un file; modifica di dati temporanei.

I layer dell'immagine sottostante non vengono modificati. Docker utilizza un meccanismo chiamato **Copy-On-Write**. Se il container deve modificare un file presente in un layer Read-Only, Docker copia quel file nel layer Read-Write del container e applica la modifica sulla copia. Così che ogni container conserva le proprie modifiche e l'immagine originale rimane invariata. Quando più container vengono creati dalla stessa Docker Image, condividono gli stessi layer Read-Only. Ogni container possiede però un proprio layer Read-Write indipendente. Quindi:

- la parte statica è condivisa;

- le modifiche di runtime sono private per ogni container.

Quando il container viene eliminato, anche il suo layer Read-Write viene eliminato. Di conseguenza, tutte le modifiche salvate soltanto al suo interno vengono perse. L'immagine invece rimane invariata e può essere nuovamente utilizzata per creare altri container.

Per conservare i dati anche dopo l'eliminazione o la ricreazione del container, Docker offre tre soluzioni principali:

- **VOLUMI** → aree del file system dell'host gestite direttamente da Docker.
Il container accede al volume tramite un punto di montaggio, ma i dati sono memorizzati fuori dal container. Se il container viene eliminato, il volume continua a esistere.
- BIND MOUNTS
- TMPFS MOUNTS

Un **Dockerfile** è un file di testo contenente le istruzioni necessarie per costruire una Docker Image. Descrive in modo ripetibile l'ambiente dell'applicazione. In particolare specifica: l'immagine di base; il codice da copiare; le dipendenze da installare; i comandi di configurazione; il comando da eseguire all'avvio.

Docker Engine legge il Dockerfile ed esegue le istruzioni in sequenza, creando i layer dell'immagine.

Oltre a dockerfile, un altro strumento è DOCKER COMPOSE. Utilizzato per **definire, configurare e gestire applicazioni composte da più container**. Per effettuare il deployment di app multi-container (app con + servizi che devono lavorare insieme).

Quando un'applicazione è costituita da più componenti (ad esempio frontend, backend e database), avviarli uno alla volta con i comandi Docker diventerebbe lungo e soggetto a errori. Docker Compose permette invece di descrivere tutta l'applicazione in un unico file di configurazione e di avviarla con un solo comando.

Le configurazioni possono essere scritte sia in **YAML** sia in **JSON**, anche se nella pratica quasi sempre viene utilizzato YAML perché è molto più leggibile. Il cuore di Docker Compose è il file **docker-compose.yml**. Si tratta di un file di configurazione che descrive tutti gli elementi necessari all'applicazione. In particolare definisce:

- i servizi (services) (componente app, corrisponde ad un container)
- le reti (networks); (per comunicazione tra servizi)
- i volumi (volumes) (per rendere persistenti i dati, visto che in container sono effimeri).

Docker Compose legge questo file e crea automaticamente tutti i container necessari. Non dobbiamo quindi creare manualmente ogni container. Possiamo

specificare come parametri anche immagine Docker da usare, porte da esporre, variabili d'ambiente. Docker compose legge automaticamente il file docker-compose.yml con comando `docker compose up` e crea automaticamente container, reti, volumi, collega tutto e avvia i container.

🔗 Le **Environment Variables** permettono di configurare l'applicazione senza modificare il codice, supportando ambienti diversi (dev, test e produzione).

🔗 Per informazioni sensibili è preferibile utilizzare i **Docker Secrets**, che vengono cifrati e montati come file nel percorso `/run/secrets/<nome_secret>`, risultando più sicuri rispetto alle normali variabili d'ambiente.

Gli **anchor** (ancore) sono una funzionalità di **YAML** che permette di **definire un valore una sola volta e riutilizzarlo in più punti del file**, evitando duplicazioni.

In pratica:

- **Anchor (&)** → assegna un nome a un valore.
- **Alias (*)** → richiama quel valore dove serve

La **Continuous Integration (CI)** è una metodologia che integra continuamente le modifiche del codice in un repository condiviso. Dopo ogni integrazione vengono eseguiti automaticamente **build e test**, così da individuare e correggere gli errori il prima possibile, migliorando la qualità del software.

La **Continuous Delivery/Deployment (CD)** automatizza il processo di rilascio del software, dalla build ai test fino al deployment, permettendo di distribuire nuove versioni in modo rapido e sicuro.

Continuous Delivery

Il software viene compilato, testato e preparato automaticamente, ma **prima del rilascio in produzione è necessaria un'approvazione manuale**.

Continuous Deployment

L'intero processo è completamente automatico: **ogni modifica che supera i test viene distribuita immediatamente in produzione**, senza intervento umano.

Differenza principale: la Continuous Delivery richiede un'approvazione manuale finale, mentre la Continuous Deployment esegue il rilascio automaticamente.

CI/CD → aumenta velocità di sviluppo e rilascio, migliore collaborazione tra sviluppatore, aumenta qualità sw grazie a test automatici e riduce rischi di errori in produzione.

AGILE

IDEE PRINCIPALI:

1. **Adattabilità e gestione del cambiamento:** Capacità di rispondere rapidamente alle modifiche dei requisiti, anche se queste si presentano in fasi avanzate dello sviluppo. Si promuove la semplicità.
2. **Rilascio incrementale e frequente:** Consegna continua e ravvicinata nel tempo (da poche settimane a un paio di mesi) di software funzionante e di valore per il cliente.
3. **Centralità delle persone e della collaborazione:** Focus su team motivati, auto-organizzati e sulla comunicazione diretta (preferibilmente faccia a faccia). Focus su soddisfazione cliente.
4. **Miglioramento continuo:** Riflessione periodica del team sulle proprie prestazioni per ottimizzare l'efficacia e i processi.

Dimensione	Approccio Tradizionale	Approccio Agile
Planning	Si concentra su una pianificazione iniziale estesa e dettagliata (<i>upfront planning</i>).	Si focalizza sulla semplicità (massimizzare il lavoro non svolto) e sull'evoluzione dei requisiti attraverso team auto-organizzati.
Release Frequency	Segue processi rigidi e, tipicamente, rilasci non frequenti basati sul piano iniziale.	Rilasci frequenti, continui e a intervalli ridotti (da poche settimane a due mesi).
Response to Change	Segue processi rigidi che rendono difficile o limitato l'adattamento ai cambiamenti.	Accoglie positivamente i cambiamenti dei requisiti in qualsiasi momento per garantire un vantaggio competitivo al cliente.
Customer Involvement	Coinvolgimento limitato o legato principalmente alle fasi di definizione iniziali dei processi rigidi.	Collaborazione stretta, quotidiana e continua tra il cliente/stakeholder e il team di sviluppo per tutto il progetto.

GOAL: To respond quickly to change and deliver value incrementally

SCRUM: **Scrum (fa largo uso di user stories)** è un framework (cioè un insieme di regole, ruoli e pratiche) utilizzato per gestire il lavoro all'interno di un progetto in modo agile. Invece di pianificare tutto l'intero progetto all'inizio e consegnare il prodotto solo alla fine (come si faceva nei metodi tradizionali), Scrum divide il lavoro in **piccoli cicli ripetitivi di breve durata** chiamati **Sprint** (che durano di solito da 1 a 4 settimane).

RUOLI:

- **PRODUCT OWNER:** È la persona che fa da ponte tra l'esterno (i clienti/utenti) e il team di sviluppo. Il suo compito principale è massimizzare il valore di business del prodotto decidendo *cosa* deve essere costruito e *quando*. Per farlo, controlla e stabilisce l'ordine di priorità degli elementi nel **Product Backlog** e lavora a stretto contatto con gli stakeholder per assicurarsi che i bisogni dell'utente finale siano compresi chiaramente dal team.
- **SCRUM MASTER:** Gestisce il team, agisce come un coach, non come un boss. **RISOLUTORE DI CONFLITTI**, deve far funzionare al meglio le cose.
- **TEAM MEMBERS:** gruppi di 5-9 persone che collaborano attivamente e si auto-organizzano.

SCRUM SPRINT: Lo Sprint è l'**iterazione di base** dell'approccio Scrum e dura da **1 a 4 settimane**. Il suo scopo principale è stabilire un periodo di tempo fisso per produrre un frammento di **software funzionante**, che possa essere mostrato e revisionato alla fine del ciclo. Durante questo periodo, la data di fine e i deliverable concordati dal team non cambiano.

Attività Principali di Scrum

- **Sprint Planning (Pianificazione):** Riunione iniziale in cui si stabilisce **cosa fare** (il Product Owner propone le storie e il team decide quali accettare) e **come farlo** (le storie vengono scomposte in task che richiedono al massimo mezza giornata di lavoro), creando così lo *Sprint Backlog*.
- **Daily Scrum (Stand-Up Meeting):** Un incontro quotidiano molto breve (massimo **15 minuti**) in cui i soli membri del team di sviluppo, rimanendo in piedi, si aggiornano su: **cosa è stato fatto**, **cosa si farà** e quali sono i **problemi od ostacoli** correnti.
- **Story Time (Backlog Refinement):** Sessione dedicata alla revisione e alla stima delle storie future, al chiarimento dei requisiti e alla scomposizione delle storie grandi, con l'obiettivo di iniziare lo sprint successivo con elementi piccoli e ben compresi.
- **Sprint Review:** Attività di fine ciclo per analizzare, mostrare e revisionare il lavoro completato e il software prodotto. Analisi di ciò che si è prodotto, vedendo anche la reazione degli stakeholder.
- **Sprint Retrospective:** Sessione finale in cui il team riflette sull'iterazione appena conclusa per valutare la propria efficacia e regolare di conseguenza il proprio comportamento.

Una **Scrum Board** è uno strumento visivo utilizzato dal team per monitorare lo stato di avanzamento delle attività (divise tipicamente in *Da fare*, *In corso* e *Fatto*) durante lo Sprint.

- ✚ **Product Backlog:** Contiene tutto ciò che consumerà le risorse del team. È composto dai **PBI (Product Backlog Items)**, che vengono ordinati e valutati con un punteggio di rilevanza relativa (*ranking* per dare priorità).
- ✚ **User Story ed Epic:** I PBI sono spesso *user stories*, ovvero frammenti gestibili di requisiti che seguono **il principio INVEST (Indipendenti, Negoziabili, Di valore, Stimabili, Piccoli, Testabili)**. Descrive le good product Backlog items properties.
- ✚ **Perché usarlo:** È ideale in ambienti moderni (come startup, grandi tech e team di prodotti digitali) in cui i requisiti cambiano rapidamente. Permette di rilasciare funzionalità più spesso, reagire ai feedback, ridurre i rischi di sviluppo e migliorare continuamente i prodotti.
- ✚ **Svantaggi principali:** richiede forte comunicazione e disciplina, è difficile da scalare in organizzazioni molto grandi, e i continui cambiamenti possono creare instabilità (portando a modifiche continue su ciò che è già stato fatto). Inoltre, è complesso stimare costi e tempi, la documentazione rischia di ridursi troppo e i team devono essere esperti e autonomi.
- ✚ **Rischi di Business:** Si riscontrano spesso requisiti mal definiti (non chiari), mancanza di una visione architeturale a lungo termine dovuta al lavoro in caselle temporali ridotte, *scope creep* per via dei troppi cambiamenti, e l'uso distorto di Agile inteso erroneamente come "assenza di processo".
- ✚ Se una storia si rivela troppo grande prende il nome di **Epic** e viene raffinata dividendola in più storie

EXTREME PROGRAMMING: XP

- ✚ **Comunicazione:** Essenziale per evitare problemi nel progetto derivanti dalla mancanza di dialogo all'interno del team.
- ✚ **Semplicità:** Focalizzarsi sul progettare la cosa più semplice che possa funzionare concretamente ("progetta per l'oggi, non per il domani").
- ✚ **Feedback:** Ottenuto mettendo il sistema in produzione il prima possibile e scrivendo costantemente i casi di test.
- ✚ **Coraggio:** Fare la cosa giusta (nel design, nei casi d'uso, ecc.) senza temere il fallimento.

TEST-DRIVEN DEVELOPMENT (TDD): I test automatizzati devono essere scritti **prima del codice di produzione** e mantenuti costantemente in esecuzione. I test si dividono in **Unit Test** (scritti dagli sviluppatori, devono passare al 100%) e **Feature/Acceptance Test** (definiti insieme al cliente per mostrare l'avanzamento del progetto rispetto alle user story). C'è una forte enfasi sui **test di regressione**, che devono essere eseguiti continuamente per l'intero sistema.

PAIR PROGRAMMING: Ogni singola funzionalità deve essere sviluppata da **due persone davanti allo stesso computer**. Mentre la prima persona implementa materialmente il codice, la seconda pensa in modo strategico a potenziali miglioramenti, casi di test ed

eventuali problemi. Le coppie cambiano continuamente, permettendo la condivisione totale della conoscenza ed evitando che vi sia un unico esperto esclusivo su una parte del sistema.

ON-SITE CUSTOMER: Il cliente diventa parte integrante del team per definire i bisogni di business.

Si parla di SMALL RELEASES: iterazioni molto veloci e cicli di sviluppo brevi (1-2 mesi), per ottenere feedback immediati.

XP ribalta alcuni approcci tradizionali in quanto:

- **Non prevede specialisti:** ogni programmatore partecipa ad architettura, design, testing e integrazione.
- **Elimina la progettazione iniziale dettagliata:** non viene fatta un'analisi o un design architettonico/infrastrutturale preventivo ed esteso (*no up-front analysis*).
- **Riduce al minimo la documentazione:** non si perde tempo a scrivere documentazione di design o implementazione al di fuori del codice e dei test stessi.